

CPSC 648: Quantum Codes and Applications to Complexity¹

Lecturer: Gaurav Mahajan

August 2024

¹Some sections of these notes are still in progress. Many of the proofs we discussed in class (on the whiteboard or from handwritten notes) haven't yet been written up in L^AT_EX.

Contents

I	Quantum Codes	3
1	Basics of Quantum Computation (Week 1)	4
1.1	Quantum bits	4
1.2	Multiple qubits	5
1.3	Single qubit quantum gates	5
1.4	Multiple qubit quantum gates	7
1.5	Quantum measurement	7
2	Introduction to Quantum Codes (Week 2)	9
2.1	Repetition code	9
2.2	Quantum noise	10
2.3	Fixing bit flip errors	11
2.4	Fixing phase errors	12
2.5	9-qubit Shor code	13
3	Knill-Laflamme Conditions (Week 3)	15
3.1	Quantum noise model	15
3.2	Quantum error-correcting code	16
3.3	Knill-Laflamme conditions	18
3.4	Distance of a code	20
4	Stabilizer Codes (Week 4)	22
4.1	Pauli group	22
4.2	Stabilizer codes	23
4.3	Dimension of stabilizer code	23
4.4	Distance of stabilizer code	24
4.5	Logical operators	26
5	Calderbank-Shor-Steane codes (Week 5)	28
5.1	Classical linear codes	28
5.2	Distance of classical linear codes	29
5.3	CSS code	30
5.4	Distance of CSS code	31

6	Toric Code (Week 6)	32
6.1	Stars and plaquettes	32
6.2	Closed loop and boundary	33
6.3	Distance of toric code	36
7	Chain Complexes (Week 7)	39
7.1	Chain Complexes	39
7.2	Homology of a chain complex	42
7.3	Dual chain	43
8	Hypergraph Product Codes (Week 8)	46
8.1	Classical codes as chain complexes	46
8.2	Hypergraph product code	48
9	Tanner Codes (Week 8)	52
9.1	Distance of Tanner Codes	53
9.2	Expander graphs	54
10	Quantum Tanner Codes	57
10.1	Left-Right Cayley Complex	57
10.2	Quantum Tanner Code	59
10.3	Dimension of the code	62
11	Distance of Quantum Tanner Code	64
11.1	Product expansion	65
11.2	Minimal representation	66
11.3	Minimum Distance	68
12	Floquet codes (Beginner's Guide)	73
12.1	Learning Objectives/Author's Disclaimer	73
12.2	Background	73
12.3	The Honeycomb Floquet Code	74
12.4	Instantaneous Toric Code Equivalence	77
12.5	Conclusion	78

Part I

Quantum Codes

Chapter 1

Basics of Quantum Computation (Week 1)

In this lecture, we introduce the basics of quantum computation needed for the quantum codes. We closely follow Chapter 1 and 2 from Nielsen and Chuang.

1.1 Quantum bits

We start by defining the central mathematical object, *qubit* (quantum bit), analogous to classical bit. Just as classical bit has state 0 and 1, a qubit also has states $|0\rangle$ and $|1\rangle$. The difference between bits and qubits is that it is also possible to form linear combinations of states, called *superpositions*:

$$|\psi\rangle = \alpha |0\rangle + \beta |1\rangle$$

for $\alpha, \beta \in \mathbb{C}$ and $|\alpha|^2 + |\beta|^2 = 1$. We will often forgo that qubits are physical objects, and think of state of a qubit as a vector in $\mathbb{C}^2$ with $|0\rangle$ and $|1\rangle$ forming an orthonormal basis, also known as *computational basis states*

$$|0\rangle = \begin{bmatrix} 1 \\ 0 \end{bmatrix}, \quad |1\rangle = \begin{bmatrix} 0 \\ 1 \end{bmatrix}.$$

And the general state

$$|\psi\rangle = \alpha |0\rangle + \beta |1\rangle = \begin{bmatrix} \alpha \\ 0 \end{bmatrix} + \begin{bmatrix} 0 \\ \beta \end{bmatrix} = \begin{bmatrix} \alpha \\ \beta \end{bmatrix}$$

An example of a single qubit state, that we will see often in the class, is $|+\rangle$ and $|-\rangle$ state.

$$|+\rangle = \frac{|0\rangle + |1\rangle}{\sqrt{2}}, \quad |-\rangle = \frac{|0\rangle - |1\rangle}{\sqrt{2}}$$

A crucial difference between classical bit and quantum bit is in their measurements. We can examine a classical bit to determine whether it is in state

0 or 1. However, we can not examine a qubit to determine its quantum state, i.e. the values of α and β . When we measure a qubit, we get either the result 0 with probability $|\alpha|^2$, or results 1 with probability $|\beta|^2$. This is also referred to as Born rule.

1.2 Multiple qubits

Suppose we have two qubits. Then, similar to classical bits, we will have 4 computational basis states $|00\rangle$, $|01\rangle$, $|10\rangle$ and $|11\rangle$. As an example, if we have two qubits: $|x\rangle = \alpha_0 |0\rangle + \alpha_1 |1\rangle$ and $|y\rangle = \beta_0 |0\rangle + \beta_1 |1\rangle$. Then, their joint state is given by their tensor product:

$$\begin{aligned} |x\rangle \otimes |y\rangle &= (\alpha_0 |0\rangle + \alpha_1 |1\rangle) \otimes (\beta_0 |0\rangle + \beta_1 |1\rangle) \\ &= \alpha_0 \beta_0 |0\rangle \otimes |0\rangle + \alpha_0 \beta_1 |0\rangle \otimes |1\rangle + \alpha_1 \beta_0 |1\rangle \otimes |0\rangle + \alpha_1 \beta_1 |1\rangle \otimes |1\rangle \end{aligned}$$

where we regard $|0\rangle \otimes |0\rangle$ as the computational state $|00\rangle$, etc.

$$|00\rangle = \begin{bmatrix} 1 \\ 0 \\ 0 \\ 0 \end{bmatrix} \quad |01\rangle = \begin{bmatrix} 0 \\ 1 \\ 0 \\ 0 \end{bmatrix} \quad |10\rangle = \begin{bmatrix} 0 \\ 0 \\ 1 \\ 0 \end{bmatrix} \quad |11\rangle = \begin{bmatrix} 0 \\ 0 \\ 0 \\ 1 \end{bmatrix}.$$

Couple of remarks are in order.

1. First, notice that tensor product is *not commutative*: for example $|0\rangle \otimes |1\rangle \neq |1\rangle \otimes |0\rangle$.
2. Another thing to keep in mind is that not all states in the multiple-qubit system are of tensor product form, namely product states. An important example of such a state is the Bell state or EPR pair,

$$\frac{|00\rangle + |11\rangle}{\sqrt{2}}.$$

Such states which can not be written in terms of a tensor product are called the *entangled states*. Later we will see many interesting properties associated to product and entangled states.

3. For n qubit system, a typical state is specified by 2^n amplitudes. For $n = 500$, 2^n is bigger than # of atoms in the universe. We can not even store these numbers in a classical computer, however, in principle, Nature manipulates such data, even for systems containing few hundred atoms.

1.3 Single qubit quantum gates

Changes occurring to a quantum state can be described using the language of quantum computation. Analogous to classical computer, a quantum computer is built from quantum gates.

We start with examining single qubit gates. For a classical bit, the only non trivial single bit gate is NOT gate. The analogous quantum NOT gate (also called X-gate) is a *linear map* which maps state $|0\rangle$ to state $|1\rangle$, and state $|1\rangle$ to state $|0\rangle$.

Definition 1.1 (X gate). *X gate acts on the computational basis as*

$$\begin{aligned} |0\rangle &\rightarrow |1\rangle \\ |1\rangle &\rightarrow |0\rangle \end{aligned}$$

A convenient way to represent the X-gate is by its matrix form:

$$X \stackrel{\text{def}}{=} \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}$$

Using this, we can write action of X-gate on an arbitrary single qubit state $|\psi\rangle = \alpha|0\rangle + \beta|1\rangle$ as

$$|\psi\rangle \rightarrow X|\psi\rangle = \alpha|1\rangle + \beta|0\rangle$$

In a circuit diagram, this looks like

$$|\psi\rangle \text{ --- } \boxed{X} \text{ --- } X|\psi\rangle$$

More generally, what properties does a linear map $U : \mathbb{C}^2 \rightarrow \mathbb{C}^2$ need to satisfy to be a valid quantum gate? The only restriction on U is that it must map quantum states to quantum states i.e. for $\langle\psi|\psi\rangle = 1$, U should satisfy

$$(U|\psi\rangle)^\dagger U|\psi\rangle = \langle\psi|U^\dagger U|\psi\rangle = 1$$

This enforces U to be unitary i.e. $U^\dagger U = I$, which is the only constraint!

So, unlike classical case (where only trivial gate is NOT gate), there are many non-trivial single qubit gates.

Definition 1.2 (Phase gate). *Z gate acts on the computational basis as*

$$\begin{aligned} |0\rangle &\rightarrow |0\rangle \\ |1\rangle &\rightarrow -|1\rangle \end{aligned}$$

Observation 1.3 (Pauli Matrices). *The matrices X , Z and $Y = -iZX$ together are known as Pauli Matrices. These matrices are observables:= i.e matrices O which are Hermitian and square to identity i.e. $O^2 = I$.*

Another important gate, Hadamard, transforms from X -basis to Z -basis.

Definition 1.4 (Hadamard gate). *H gate acts on the computational basis as*

$$\begin{aligned} |0\rangle &\rightarrow |+\rangle \\ |1\rangle &\rightarrow |-\rangle \end{aligned}$$

1.4 Multiple qubit quantum gates

Unlike classical gates, unitary quantum gates are always invertible. For example, if we consider XOR gate: (for input A and B, outputs $A \oplus B$), we can not recover the inputs A and B, given its output: there is a loss of information. A typical 2-qubit gate is a CNOT gate, a generalization of the XOR gate.

Definition 1.5 (CNOT gate). *CNOT gate is described as*

$$\begin{array}{lcl} \text{control qubit } |a\rangle & \text{---} \bullet \text{---} & |a\rangle \\ \text{target qubit } |b\rangle & \text{---} \oplus \text{---} & |a + b \pmod{2}\rangle \end{array}$$

which acts on the computational basis as

$$\begin{aligned} |00\rangle &\rightarrow |00\rangle \\ |01\rangle &\rightarrow |01\rangle \\ |10\rangle &\rightarrow |11\rangle \\ |11\rangle &\rightarrow |10\rangle \end{aligned}$$

1.5 Quantum measurement

We now describe what happens when an experimentalist and their equipment performs a measurement on the system. We first define quantum measurements.

Quantum measurements are described by a collection $\{M_m\}$ of measurement operators where the index m refers to the measurement outcomes. If the state of the quantum state is $|\psi\rangle$ immediately before the measurement, then the probability that result m occurs is given by

$$p(m) = \langle \psi | M_m^\dagger M_m | \psi \rangle ,$$

and the state of the system after the measurement is

$$\frac{M_m |\psi\rangle}{\sqrt{p(m)}} .$$

The measurement operators should satisfy the completeness equation

$$\sum_m M_m^\dagger M_m = I$$

which ensures that the outcome probabilities sum to 1

$$\sum_m p(m) = \sum_m \langle \psi | M_m^\dagger M_m | \psi \rangle = 1 .$$

Observation 1.6. *The following can be verified:*

1. In general, M_m need not be Hermitian.

2. *Non-orthogonal states can not reliably distinguished.*
3. *We can simulate measurement of a qubit in computational basis using*

$$M_0 = |0\rangle\langle 0|, \quad M_1 = |1\rangle\langle 1|$$

A subclass of measurements, namely projective measurements will be useful in measuring parity/syndromes for codewords.

Definition 1.7 (Projective measurements). *Projective (or von Neumann) measurement is described by an observable, M , a Hermitian operator on the state space. The operator has a spectral decomposition:*

$$M = \sum_m m P_m$$

where P_m is the projector onto the eigenspace of M with eigenvalue m . Again, index m refers to the measurement outcome with probability $p(m) = \langle \psi | P_m | \psi \rangle$, and the state after outcome m is given by

$$\frac{P_m |\psi\rangle}{\sqrt{p(m)}}$$

Example 1.8 (Measurement of observable Z). *Z has eigenvalue $+1$ and -1 with eigenvectors $|0\rangle$ and $|1\rangle$:*

$$P_{+1} = |0\rangle\langle 0|, P_{-1} = |1\rangle\langle 1|.$$

Chapter 2

Introduction to Quantum Codes (Week 2)

In this lecture, we will construct our first quantum error correcting code. We will borrow a lot of terminology and conceptual ideas from the classical error-correction literature. Therefore, we start by reviewing a basic classical error correcting code, known as repetition code.

2.1 Repetition code

In classical world, our data consists of bits $b \in \{0, 1\}$. Suppose we wish to send a bit from one location to another through a noisy classical channel. The effect of noise in the channel is to flip the bit being transmitted with probability $p > 0$. To deal with this noise, we encode bit b by adding redundant bits.

Definition 2.1 (Repetition Code). *Repetition code encodes $b \in \{0, 1\}$ into 3 bits.*

$$0 \rightarrow 000$$

$$1 \rightarrow 111$$

The bit strings 000 and 111 are referred to as *codewords*, and also sometimes referred to as the *logical 0* and *logical 1*, since they play the role of 0 and 1 respectively.

We now send all three bits 000 through the channel. Suppose our codeword experiences a single bit flip error, and the output of the channel is 010:

$$000 \xrightarrow{\text{noise}} 010$$

We can correct this bit flip by looking at the majority of the three bits. Since, only one bit has been flipped, this will give us the correct value for the three bits:

$$000 \xrightarrow{\text{noise}} 010 \xrightarrow{\text{majority}} 000$$

On the other hand, if there were 2 or more bit flips, this scheme would fail

$$000 \xrightarrow{\text{lots of noise}} 011 \xrightarrow{\text{majority}} 111 \neq 000$$

Hence, the 3-bit repetition code allows us to correct one bit flip error, and fails to correct 2 or more bit flips. This means we could use repetition code to get reliability in channels where error probability p is small. These channels mostly see at most 1 bit flip error, since the probability of seeing 2 or more bit flip errors scales as $O(p^2)$.

2.2 Quantum noise

Classically, bit-flip is the only possible type of noise. But a quantum state is susceptible to many more types of noise. Let's look at few examples of quantum error.

Definition 2.2 (Bit flip error). *A bit flip error is described by the unitary*

$$X = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}$$

which acts on the computational basis as

$$X|0\rangle = |1\rangle$$

$$X|1\rangle = |0\rangle$$

Definition 2.3 (Phase flip error). *A phase flip error is described by the unitary*

$$Z = \begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix}$$

which acts on the computational basis as

$$Z|0\rangle = |0\rangle$$

$$Z|1\rangle = -|1\rangle$$

Observation 2.4. *Phase flip error acts like a bit flip error in the Hadamard basis since $Z = HXH$. We check by direct computation.*

$$\begin{aligned} Z|+\rangle &= Z \frac{|0\rangle + |1\rangle}{\sqrt{2}} = \frac{|0\rangle - |1\rangle}{\sqrt{2}} = |-\rangle \\ Z|-\rangle &= Z \frac{|0\rangle - |1\rangle}{\sqrt{2}} = \frac{|0\rangle + |1\rangle}{\sqrt{2}} = |+\rangle \end{aligned}$$

In today's lecture, we will consider a simplified model of quantum noise where only *bit flip* and *phase flip* errors can occur. We will see in the lecture tomorrow that this is without loss of generality, because Pauli errors $\{I, X, Z, Y = iXZ\}$ form an orthogonal basis for the Hilbert space of all 2×2 Hermitian matrices. This means, if a quantum code corrects for Pauli errors¹, it can correct arbitrary errors for free.

¹since Y error is just a phase flip error and a bit flip error, we actually only need to worry about X and Z errors

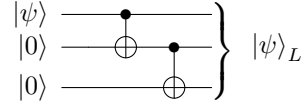
2.3 Fixing bit flip errors

We now describe a 3-qubit quantum code that protects against bit flip errors. This was first investigated by Asher Peres in 1985.

Definition 2.5 (3-qubit bit flip code). *We define the code on the standard basis, and then extend via linearity.*

$$\begin{aligned} |0\rangle &\rightarrow |000\rangle \\ |1\rangle &\rightarrow |111\rangle \\ |\psi\rangle = a|0\rangle + b|1\rangle &\rightarrow a|000\rangle + b|111\rangle = |\psi\rangle_L \end{aligned}$$

How do we implement this code? Since we can not clone quantum states, we can not go from $|0\rangle$ to $|000\rangle$. To implement this, we first add two ancilla qubits. To familiarize us with quantum circuits, here is a circuit which implements this code:



The gate used above is a CNOT gate (Definition 1.5).

We encode $|\psi\rangle = a|0\rangle + b|1\rangle$, and send the qubits $|\psi\rangle_L = a|000\rangle + b|111\rangle$ through the quantum channel. Now, suppose there is a bit flip error, say on the second qubit.

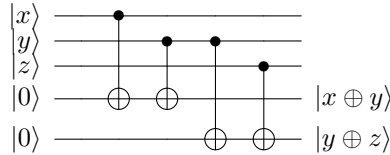
$$(I \otimes X \otimes I) \cdot |\psi\rangle_L = a|010\rangle + b|101\rangle = |\tilde{\psi}\rangle_L$$

To detect a bit flip error, a natural strategy could be to measure the 3 qubits in the standard basis. This measurement gives string 010 with probability $|a|^2$, and string 101 with probability $|b|^2$. Using this, we can detect that a bit flip error occurred. However, in the process, we have collapsed our state $|\tilde{\psi}\rangle_L$, and therefore, we can not recover the original state.

The issue is that we have measured too much. We should try to measure only “Has a bit flip error occurred, and on which qubits?” i.e. (1) Measure parity of first and second qubits, and (2) Measure parity of second and third qubits.

Both parities are 1 and 1, for both $|010\rangle$ and $|101\rangle$, so measuring it does not collapse the state. In other words, we perform the following circuit on $|\tilde{\psi}\rangle_L$.

Here is a circuit which computes the parity of 1st and 2nd qubit, and the parity of 2nd and 3rd qubit.



The circuit acts on $|\tilde{\psi}\rangle_L = a|010\rangle + b|101\rangle$ as follows:

$$\begin{aligned} |\tilde{\psi}\rangle_L \otimes |00\rangle &= a|010\rangle \otimes |00\rangle + b|101\rangle \otimes |00\rangle \\ &\xrightarrow[\text{parity on 2 ancilla}]{\text{circuit computing}} a|010\rangle \otimes |11\rangle + b|101\rangle \otimes |11\rangle \\ &= |\tilde{\psi}\rangle_L \otimes |11\rangle \end{aligned}$$

By measuring the last two registers in computational basis (which gives the measurement outcome 11), we can infer that the bit flip error occurred on 2nd qubit. The measurement outcome 11 is called syndrome, and it gives the required information about the error, i.e. in this case, a bit flip occurred on the second qubit. We can infer other errors from the syndrome as follow:

Syndrome	Error
00	no error
01	bit flip on 3rd qubit
10	bit flip on 1rd qubit
11	bit flip on 2rd qubit

In our case, we can fix the error by applying second bit flip.

$$(I \otimes X \otimes I) \cdot |\tilde{\psi}\rangle_L = |\psi\rangle_L$$

Note that this code can not fix two bit flip errors. Moreover, it can not fix phase flip errors.

$$\begin{aligned} (Z \otimes I \otimes I) \cdot |\psi\rangle_L &= (Z \otimes I \otimes I)(a|000\rangle + b|111\rangle) \\ &= a|000\rangle - b|111\rangle \end{aligned}$$

We can not even detect this error because it is an encoding of state $a|0\rangle - b|1\rangle$.

Observation 2.6 (Measuring parity). *Here is another interpretation for the parity measurement. We can measure “parity of 1st and 2nd qubit” and “parity of 2nd and 3rd qubit” by measurements of observables Z_1Z_2 and Z_2Z_3 . Here the notation Z_iZ_j means Z gate applies on i th and j th qubit, and I on all remaining qubits.*

In our case, Z_1Z_2 has the spectral decomposition

$$Z_1Z_2 = (|00\rangle\langle 00| + |11\rangle\langle 11|) \otimes I - (|01\rangle\langle 01| + |10\rangle\langle 10|) \otimes I$$

Notice how the eigenvalues characterize the parity, and therefore, measuring observable Z_1Z_2 on $|\tilde{\psi}\rangle_L = a|010\rangle + b|101\rangle$ does not change the state, and outputs -1 and identifies the syndrome.

2.4 Fixing phase errors

We now describe a code which corrects phase flip errors. The main observation to fix phase errors is that X and Z errors are switched by Hadamard transform

$$X = HZH$$

Definition 2.7 (3-qubit phase flip code). *We define the code on the standard basis, and then extend via linearity.*

$$\begin{aligned} |0\rangle &\rightarrow |+++ \rangle \\ |1\rangle &\rightarrow |-- - \rangle \\ |\psi\rangle &= a|0\rangle + b|1\rangle \rightarrow a|+++ \rangle + b|-- - \rangle = |\psi\rangle_L \end{aligned}$$

Let's see what happens when there is a phase flip error, say on second qubit

$$Z_2 |\psi\rangle_L = a|+-+ \rangle + b|-+- \rangle = |\tilde{\psi}\rangle_L$$

Now, we can locate the error just like before, by first transforming to $|0\rangle, |1\rangle$ bases. The circuit acts on $|\tilde{\psi}\rangle_L = a|+-+ \rangle + b|-+- \rangle$ as follows:

$$\begin{aligned} |\tilde{\psi}\rangle_L \otimes |00\rangle &= a|+-+ \rangle \otimes |00\rangle + b|-+- \rangle \otimes |00\rangle \\ &\xrightarrow[\text{first 3 qubits}]{\text{Hadamard on}} a|010\rangle \otimes |00\rangle + b|101\rangle \otimes |00\rangle \\ &\xrightarrow[\text{parity on 2 ancilla}]{\text{circuit computing}} a|010\rangle \otimes |11\rangle + b|101\rangle \otimes |11\rangle \\ &\xrightarrow[\text{first 3 qubits}]{\text{Hadamard on}} |\tilde{\psi}\rangle_L \otimes |11\rangle \end{aligned}$$

In other words, the syndrome measurement can be performed by measuring the observable $H^{\otimes 3} Z_1 Z_2 H^{\otimes 3} = X_1 X_2$ and $H^{\otimes 3} Z_2 Z_3 H^{\otimes 3} = X_2 X_3$. In our case, we can fix the error by applying phase flip to the second qubit.

2.5 9-qubit Shor code

We now discuss a simple code, Shor's 9-qubit code, which protects against arbitrary single qubit errors! This code is an example of code concatenation: a combination of 3-qubit bit flip code and 3-qubit phase flip code. We first encode with phase flip code: $|0\rangle \rightarrow |+++ \rangle$ and $|1\rangle \rightarrow |-- - \rangle$, and then encode each of these qubits with bit flip code: $|+\rangle \rightarrow (|000\rangle + |111\rangle)/\sqrt{2}$ and $|-\rangle \rightarrow (|000\rangle - |111\rangle)/\sqrt{2}$.

Definition 2.8 (9-qubit Shor code). *We define the code on the standard basis, and then extend via linearity.*

$$\begin{aligned} |0\rangle &\rightarrow \left(\frac{|000\rangle + |111\rangle}{\sqrt{2}} \right)^{\otimes 3} \stackrel{\text{def}}{=} |0\rangle_L \\ |1\rangle &\rightarrow \left(\frac{|000\rangle - |111\rangle}{\sqrt{2}} \right)^{\otimes 3} \stackrel{\text{def}}{=} |1\rangle_L \\ |\psi\rangle &= a|0\rangle + b|1\rangle \rightarrow a|0\rangle_L + b|1\rangle_L = |\psi\rangle_L \end{aligned}$$

Now, suppose a bit flip error occurs on the second qubit of the first block

$$\begin{aligned}
 I \otimes X \otimes I^{\otimes 7} |\psi\rangle_L &= (I \otimes X \otimes I^{\otimes 7}) \cdot (a|0\rangle_L + b|1\rangle_L) \\
 &= a \left(\frac{|010\rangle + |101\rangle}{\sqrt{2}} \right) \cdot \left(\frac{|000\rangle + |111\rangle}{\sqrt{2}} \right)^{\otimes 2} \\
 &\quad + b \left(\frac{|010\rangle - |101\rangle}{\sqrt{2}} \right) \cdot \left(\frac{|000\rangle - |111\rangle}{\sqrt{2}} \right)^{\otimes 2} \\
 &= |\tilde{\psi}\rangle_L
 \end{aligned}$$

We can now apply the circuit from bit flip detection on first 3 qubits to identify the error. In other words, we measure Z_1Z_2 and Z_2Z_3 , and in our case, it will give measurement outcome 11.

Instead, if we had a phase flip error occur on the second qubit of the first block

$$\begin{aligned}
 I \otimes Z \otimes I^{\otimes 7} |\psi\rangle_L &= (I \otimes Z \otimes I^{\otimes 7}) \cdot (a|0\rangle_L + b|1\rangle_L) \\
 &= a \left(\frac{|000\rangle - |111\rangle}{\sqrt{2}} \right) \cdot \left(\frac{|000\rangle + |111\rangle}{\sqrt{2}} \right)^{\otimes 2} \\
 &\quad + b \left(\frac{|000\rangle + |111\rangle}{\sqrt{2}} \right) \cdot \left(\frac{|000\rangle - |111\rangle}{\sqrt{2}} \right)^{\otimes 2} \\
 &= |\tilde{\psi}\rangle_L
 \end{aligned}$$

To identify the phase flip error, we measure $X_1X_2X_3X_4X_5X_6$ and $X_4X_5X_6X_7X_8X_9$. $X_1X_2X_3$ looks at the first block and returns +1 for $|000\rangle + |111\rangle$ and -1 for $|000\rangle - |111\rangle$. Similarly $X_4X_5X_6$ acts on the second block and therefore, $X_1X_2X_3X_4X_5X_6$ compares the sign of first two blocks and returns 1 if they are same and -1 otherwise.

Chapter 3

Knill-Laflamme Conditions (Week 3)

In last lecture, we built our first quantum error correction code, which is able to correct arbitrary 1-qubit errors. In general, however, once you discover a new quantum code, it's unclear: What errors are correctable using this quantum code?

In today's lecture, we will build a general framework to study this question. In particular, we will prove that a quantum code corrects a set of errors if and only if a set of conditions, called Knill-Laflamme conditions, are satisfied. Even though these conditions do not guarantee that good quantum codes exist, these conditions will be very useful in analyzing the properties of our quantum codes. We follow very closely the nice exposition on this topic from [1].

3.1 Quantum noise model

In last lecture, we only looked at bit flip and phase flip errors. So, let's first define our general model of noise. We will need an equivalent way of representing states: density matrices. The density matrix ρ associated to a quantum state $|\psi\rangle$ is given by $\rho = |\psi\rangle\langle\psi|^1$.

The general model of noise is that it is a quantum channel, denoted by $N(\cdot)$, and we assume its operator representation is given by

$$N(\rho) = \sum_k E_k \rho E_k^\dagger \quad \text{where} \quad \sum_k E_k E_k^\dagger = I$$

Each of these E_i 's represent an "error" which can be applied to this state.

¹Two different states which have same density matrix are information theoretically indistinguishable. An example is equal mixtures of $|0\rangle, |1\rangle$ and $|+\rangle, |-\rangle$

Operationally, we can think of this noise channel as outputting state

$$\frac{E_k \rho E_k^\dagger}{\text{Tr}(E_k \rho E_k^\dagger)}$$

with probability $\text{Tr}(E_k \rho E_k^\dagger)$. A few examples of single qubit noise channels.

Example 3.1 (Generalized Pauli Channel).

$$N(\rho) = p_x X \rho X^\dagger + p_y Y \rho Y^\dagger + p_z Z \rho Z^\dagger + (1 - p_x - p_y - p_z) \rho$$

This channel applies X, Y, Z errors with probability p_x, p_y, p_z respectively. Here all errors are unitary, and the probability of error does not depend on the state.

Example 3.2 (Measurement channel). *An example where noise might perform measurement on your state.*

$$N(\rho) = (|0\rangle\langle 0|)\rho(|0\rangle\langle 0|)^\dagger + (|1\rangle\langle 1|)\rho(|1\rangle\langle 1|)^\dagger$$

Here, the errors are not unitary, $E_0 = |0\rangle\langle 0|$ and $E_1 = |1\rangle\langle 1|$. Moreover, interestingly, the probability of error depends on the state, for state $|\psi\rangle = a|0\rangle + b|1\rangle$, the probability of error E_0 is

$$\text{Tr}(E_0 \rho E_0^\dagger) = |a|^2$$

Example 3.3 (Amplitude damping channel).

$$N(\rho) = E_0 \rho E_0^\dagger + E_1 \rho E_1^\dagger$$

where

$$E_0 = \begin{bmatrix} 1 & 0 \\ 0 & \sqrt{1-\gamma} \end{bmatrix}, \quad E_1 = \begin{bmatrix} 0 & \sqrt{\gamma} \\ 0 & 0 \end{bmatrix}$$

If the state is in $|0\rangle$ state, then nothing happens, but if the state is in $|1\rangle$ state, then with probability γ , it goes to $|0\rangle$ state.

In this class, we care about n-qubit channels. The basic channel that we will look at is one which is made up of single qubit channels.

Definition 3.4. ξ is an independent channel if we can write ξ as $\xi_1 \otimes \xi_2 \otimes \dots \otimes \xi_n$ where $\{\xi_i\}$ are single qubit channels.

Example 3.5. $\xi = N^{\otimes n}$ where $N(\rho) = (1-p) \cdot \rho + pZ\rho Z^\dagger$

3.2 Quantum error-correcting code

Next, we formally define a quantum error correcting code. We will view a quantum error-correcting code as a mapping from a smaller Hilbert space to a subspace of a larger Hilbert space.

Definition 3.6 (Quantum Error-Correcting Code). A QECC is defined to be a subspace $C \subset \mathcal{H}_{\text{physical}}$ with a unitary encoding map $U : \mathcal{H}_{\text{logical}} \rightarrow C$.

Throughout this lecture, we will consider the following setup: we map k logical qubits to n physical qubits

$$\mathcal{H}_{\text{logical}} = (\mathbb{C}^2)^{\otimes k}, \quad \mathcal{H}_{\text{physical}} = (\mathbb{C}^2)^{\otimes n}$$

and the quantum error correcting code is given by

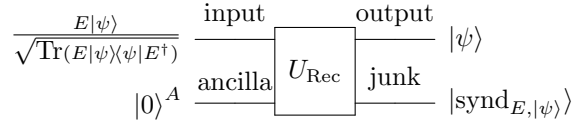
$$C = \text{span} \{|\bar{x}\rangle\} \quad \text{where } |\bar{x}\rangle = U(|x\rangle \otimes |0\rangle^{\otimes n-k}) \text{ and } x \in \{0, 1\}^k$$

We now define what it means for an error to be correctable by a quantum code.

Definition 3.7. We say a set of errors $\mathcal{E}$ is corrected by the code C if there exists a quantum algorithm $\text{Rec}(\cdot)$ such that for all $E \in \mathcal{E}$ and states $|\psi\rangle \in C$,

$$\text{Rec} \left(\frac{E|\psi\rangle}{\sqrt{\text{Tr}(E|\psi\rangle\langle\psi|E^\dagger)}} \right) = |\psi\rangle$$

Note that if you correct the set of errors $\mathcal{E}$, then you can also correct noise channel whose operators are from $\mathcal{E}$. Operationally, we require that there is a unitary U_{Rec} , such that if the input register has state $E_i|\psi\rangle$, and the input ancilla register is set to $|0\rangle^A$, then after running the recovery algorithm, the output register should have state $|\psi\rangle$, and the output ancilla qubits would contain the syndrome (just like Shor's code).



Observation 3.8. We will see soon that for QECC,

1. even though the definition of $\text{Rec}(\cdot)$ allows the syndrome to depend on the input state, in principle, the syndrome should not depend on the input state $|\psi\rangle$. This is related to preservation of information: if we can glean any information about state $|\psi\rangle$ from the syndrome, then output state can not be $|\psi\rangle$.
2. the correctable errors $\mathcal{E}$ can be defined independently of the recovery algorithm $\text{Rec}(\cdot)$ (or in other words, there is a canonical recovery algorithm).

One interesting consequence of our definition of correctable errors (and linearity), is that correctable errors form a linear subspace.

Fact 3.9. Let $\mathcal{E}$ be the set of errors on C that $\text{Rec}(\cdot)$ corrects. Then, $\mathcal{E}$ is a linear subspace (on matrices $\mathbb{C}^{2^n \times 2^n}$).

Proof. To show that the set of correctable errors $\mathcal{E}$ forms a linear subspace, we need to show

$$E_1, E_2 \in \mathcal{E} \implies (E_1 + E_2) \in \mathcal{E}.$$

This follows from linearity of U_{Rec} : for any $E_1, E_2 \in \mathcal{E}$,

$$\begin{aligned} & \frac{(E_1 + E_2) |\psi\rangle}{\sqrt{\text{Tr}((E_1 + E_2) |\psi\rangle\langle\psi| c(E_1 + E_2)^\dagger)}} \otimes |0\rangle^A \\ &= \sum_{i \in \{1,2\}} \frac{\sqrt{\text{Tr}(E_i |\psi\rangle\langle\psi| E_i^\dagger)}}{\sqrt{\text{Tr}((E_1 + E_2) |\psi\rangle\langle\psi| c(E_1 + E_2)^\dagger)}} \frac{E_i |\psi\rangle}{\sqrt{\text{Tr}(E_i |\psi\rangle\langle\psi| E_i^\dagger)}} \otimes |0\rangle^A \\ &\xrightarrow{U_{\text{Rec}}} \sum_{i \in \{1,2\}} \frac{\sqrt{\text{Tr}(E_i |\psi\rangle\langle\psi| E_i^\dagger)}}{\sqrt{\text{Tr}((E_1 + E_2) |\psi\rangle\langle\psi| c(E_1 + E_2)^\dagger)}} |\psi\rangle \otimes |\text{synd}_{E_i, |\psi\rangle}\rangle \\ &= |\psi\rangle \otimes \left(\sum_{i \in \{1,2\}} \frac{\sqrt{\text{Tr}(E_i |\psi\rangle\langle\psi| E_i^\dagger)}}{\sqrt{\text{Tr}((E_1 + E_2) |\psi\rangle\langle\psi| c(E_1 + E_2)^\dagger)}} |\text{synd}_{E_i, |\psi\rangle}\rangle \right) \end{aligned}$$

□

3.3 Knill-Laflamme conditions

We now give an algorithm independent characterization of the subspace of correctable errors. We follow the exposition and proof from John Wright's lecture notes [1]. Recall a QECC is a subspace $C \subset \text{of } \mathcal{H}_{\text{physical}}$. C corrects a set of errors $\mathcal{E}$ if there exists a recovery alg $\text{Rec}(\cdot)$ such that for all states $|\psi\rangle \in C$ and errors $E \in \mathcal{E}$,

$$\begin{array}{c} \text{Rec} \left(\frac{E |\psi\rangle}{\sqrt{\text{Tr}(E |\psi\rangle\langle\psi| E^\dagger)}} \right) = |\psi\rangle \\ \begin{array}{ccc} \frac{E |\psi\rangle}{\sqrt{\text{Tr}(E |\psi\rangle\langle\psi| E^\dagger)}} & \xrightarrow{\text{input}} & \xrightarrow{\text{output}} |\psi\rangle \\ & \text{ancilla } |0\rangle^A & \text{junk } |\text{synd}_{E, |\psi\rangle}\rangle \end{array} \\ & \boxed{U_{\text{Rec}}} \end{array}$$

We can now identify conditions such a code should satisfy.

1. Suppose we run the recovery algorithm on input state $E |\psi\rangle$ for some correctable error $E \in \mathcal{E}$ and codeword $|\psi\rangle \in C$. Then, according to definition above

$$\underbrace{U_{\text{Rec}}(E |\psi\rangle \otimes |0^A\rangle)}_{\text{Linear in } |\psi\rangle} = |\psi\rangle \otimes \underbrace{\sqrt{\text{Tr}(\langle\psi| E^\dagger E |\psi\rangle)} \cdot |\text{synd}_{E, |\psi\rangle}\rangle}_{\text{constant vector independent of } |\psi\rangle}$$

Recall the probability of applying error E on codeword $|\psi\rangle$ is given by $\text{Tr}(\langle\psi| E^\dagger E |\psi\rangle)$. But because the left hand side is linear in $|\psi\rangle$, both the

probability of applying error E , and the associated syndrome must be independent of $|\psi\rangle$ i.e. for all codewords $|\psi\rangle \in C$,

$$\text{Tr}(\langle\psi|E^\dagger E|\psi\rangle) \stackrel{\text{def}}{=} p_E \quad \text{and} \quad |\text{synd}_{E,\psi}\rangle \stackrel{\text{def}}{=} |\text{synd}_E\rangle.$$

and therefore, output of the recovery on input $E|\psi\rangle$ is

$$U_{\text{Rec}}(E|\psi\rangle \otimes |0^A\rangle) = |\psi\rangle \otimes \sqrt{p_E} \cdot |\text{synd}_E\rangle$$

2. Taking this a bit further, let's look at the overlap of the output of the recovery algorithm on 2 different codewords $|\psi_1\rangle, |\psi_2\rangle$ and errors E_1, E_2 ,

$$\begin{aligned} \langle\psi_2|E_2^\dagger E_1|\psi_1\rangle &= \underbrace{\left(\langle\psi_2|E_2^\dagger \otimes \langle 0^A|\right) U_{\text{Rec}}^\dagger}_{\text{output on } E_2|\psi_2\rangle} \cdot \underbrace{U_{\text{Rec}}(E_1|\psi_1\rangle \otimes |0^A\rangle)}_{\text{output on } E_1|\psi_1\rangle} \\ &= (\langle\psi_2| \otimes \sqrt{p_{E_2}} \cdot \langle\text{synd}_{E_2}|) \cdot (|\psi_1\rangle \otimes \sqrt{p_{E_1}} \cdot |\text{synd}_{E_1}\rangle) \\ &= \langle\psi_2|\psi_1\rangle \cdot \underbrace{\sqrt{p_{E_1}}\sqrt{p_{E_2}} \langle\text{synd}_{E_2}|\text{synd}_{E_1}\rangle}_{\text{overlap of error } E_1, E_2} \end{aligned}$$

Note that $\sqrt{p_{E_1}}\sqrt{p_{E_2}} \langle\text{synd}_{E_2}|\text{synd}_{E_1}\rangle$ is some constant dependent only on the errors, independent of codestates. And therefore, the inner product $\langle\psi_2|E_2^\dagger E_1|\psi_1\rangle$ decomposes into inner products on the codespace, and inner product on the errors.

We just showed above that these conditions are necessary for error correction. It turns out these conditions are also sufficient for error correction.

Theorem 3.10 (Knill-Laflamme Conditions). *The set of errors $\mathcal{E}$ are correctable on a code C iff for all codewords $|\psi_1\rangle, |\psi_2\rangle \in C$ and errors $E_1, E_2 \in \mathcal{E}$,*

$$\langle\psi_1|E_1^\dagger E_2|\psi_2\rangle = \langle\psi_1|\psi_2\rangle \cdot O_{E_1, E_2}$$

where O_{E_1, E_2} is some constant dependent on errors E_1, E_2 .

Before we prove these are sufficient, we first discuss few equivalent formulations for Knill-Laflamme conditions. Recall we use $|\bar{x}\rangle \in \mathcal{H}_{\text{physical}} = (\mathbb{C}^2)^{\otimes n}$ to denote the physical encoding of the logical bits $|x\rangle \in \mathcal{H}_{\text{logical}} = (\mathbb{C}^2)^{\otimes k}$.

1. Consider an orthonormal basis $\{|\bar{x}\rangle\}_{x \in \{0,1\}^k}$ of C . Then,

$$\langle\bar{x}|E_1^\dagger E_2|\bar{y}\rangle = \delta_{x,y} \cdot O_{E_1, E_2} \quad \forall E_1, E_2 \in \mathcal{E} \text{ and } x, y \in \{0,1\}^k$$

Proof. $\implies$ follows immediately.

$\impliedby$ follows by writing $|\psi_1\rangle, |\psi_2\rangle$ in the orthonormal basis, and then using linearity. $\square$

2. Let $\{E_a\}_a$ be a basis of $\mathcal{E}$. Then,

$$\langle\bar{x}|E_a^\dagger E_b|\bar{y}\rangle = \delta_{x,y} \cdot O_{E_a, E_b} \quad \forall E_a, E_b \text{ in basis and } x, y \in \{0,1\}^k \quad (3.1)$$

Proof. $\implies$ follows immediately.

$\Leftarrow$ follows by writing E_1, E_2 in the basis, and then using linearity. $\square$

3. For all $|\psi\rangle \in C$ and $E \in \mathcal{E}$,

$$\langle \psi | E^\dagger E | \psi \rangle = O_{E,E}$$

Proof. $\implies$ follows immediately.

$\Leftarrow$ follows by substituting $|\psi\rangle = (|\psi_1\rangle + |\psi_2\rangle) / \|\psi_1 + \psi_2\|_2$, and this allows us to get cross term $\langle \psi_1 | E^\dagger E | \psi_2 \rangle$ (and $\langle \psi_2 | E^\dagger E | \psi_1 \rangle$ which are equal). The rest follows similarly. $\square$

We now show how to build a recovery algorithm provided our code satisfies these conditions. We gave two proofs in the class.

TODO: Add the two proofs discussed in the class.

3.4 Distance of a code

Definition 3.11 (Weight of errors). *We will say $A_1 \otimes A_2 \cdots \otimes A_n \in L(\mathbb{C}^2)^{\otimes n}$ has weight t if all but t of the A_i 's are all I . This corresponds to errors acting on t -qubits out of n .*

Definition 3.12 (Distance of a code). *We say a code $C \subset \mathcal{H}_{\text{physical}} = (\mathbb{C}^2)^{\otimes n}$ has distance $2t + 1$ (or equivalently corrects all weight- t errors) if any of the following equivalent conditions holds:*

- $\langle \psi | E_1^\dagger E_2 | \psi \rangle = O_{E_1, E_2}$ for all weight- t errors E_1, E_2 .
- $\langle \psi | P_1^\dagger P_2 | \psi \rangle = O_{P_1, P_2}$ for all weight- t Pauli P_1, P_2 .

Proof. This is because Pauli matrices $\{I, X, Y, Z\}$ form a basis for all matrices in $L(\mathbb{C}^2)$. Moreover, weight- t $\{I, X, Y, Z\}^{\otimes n}$ form a basis for all weight- t matrices in $L(\mathbb{C}^2)^{\otimes n}$. $\square$

- $\langle \psi | P | \psi \rangle = O_P$ for all weight- $2t$ Pauli P .

Proof. This is because $P_1^\dagger P_2$ is a weight- $2t$ Pauli. $\square$

- $\langle \psi | E | \psi \rangle = O_E$ for all weight- $2t$ errors E .

Proof. Again by noting that weight- $2t$ $\{I, X, Y, Z\}^{\otimes n}$ form a basis for all weight- $2t$ matrices in $L(\mathbb{C}^2)^{\otimes n}$. $\square$

Observation 3.13. *This is consistent with our traditional definition of distance of a code. If we had a 5-repetition code with codewords 00000 and 11111, the distance would be the hamming distance between the two codewords which is 5. And this code can correct 2 bit-flip errors.*

Notation. We will use the notation $[[n, k, d]]$ -QECC to denote a quantum error correcting code which has n physical qubits, k logical qubits and distance d .

Chapter 4

Stabilizer Codes (Week 4)

We now have the theoretical framework to study quantum error correcting codes, but we still do not know many examples of such codes. Our goal in this lecture is to introduce a powerful technology, Stabilizer Codes, to construct quantum codes.

4.1 Pauli group

The key to the power of stabilizer formalism lies in its clever use of group theory. The particular group we will use is Pauli group. We first recall the Pauli matrices.

$$X = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}, \quad Z = \begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix}, \quad Y = -iZX = \begin{bmatrix} 0 & -i \\ i & 0 \end{bmatrix}$$

These matrices either commute or anti commute with each other.

$$XY = -YX, \quad YZ = -ZY, \quad ZX = -XZ$$

Definition 4.1 (Pauli Group, Pauli_n). *The n -qubit Pauli group Pauli_n is the group formed by*

$$\underbrace{\pm P_1 \otimes P_2 \otimes \dots \otimes P_n}_{\text{observables}}, \quad \underbrace{\pm i P_1 \otimes P_2 \otimes \dots \otimes P_n}_{\text{not observables}}$$

where each $P_i \in \{I, X, Y, Z\}$

We need the ± 1 and $\pm i$ factors to ensure this group is closed under matrix multiplication. Note that tensor products $P_1 \otimes P_2 \otimes \dots \otimes P_n$ and $Q_1 \otimes Q_2 \otimes \dots \otimes Q_n$ also either commute (even) or anti-commute (odd) based on number of locations with anti-commuting Paulis.

4.2 Stabilizer codes

The basic idea of the stabilizer formulation is that many quantum states can be more easily described by working with operators that “stabilize” them than by explicitly working with states itself.

Definition 4.2 (Stabilizer Code). *Let S be any subgroup of Pauli_n . Then, the stabilizer code $C[S]$ is given by*

$$C[S] \stackrel{\text{def}}{=} \{|\psi\rangle \in (\mathbb{C}^2)^{\otimes n} : P|\psi\rangle = |\psi\rangle, \forall P \in S\}$$

We call S , stabilizer group, and its elements, parity checks/stabilizers.

Fact 4.3. *For $C[S]$ to be non-empty, we need*

1. *for all $P, Q \in S$, $PQ = QP$.*

Proof. If $PQ \neq QP$, then $PQ = -QP$ (because P, Q are Pauli matrices). For any $|\psi\rangle \in C[S]$, we get

$$\begin{aligned} PQ|\psi\rangle &= P|\psi\rangle = |\psi\rangle \\ QP|\psi\rangle &= Q|\psi\rangle = |\psi\rangle \end{aligned}$$

which is only possible together with $PQ = -QP$ iff $|\psi\rangle = 0$. $\square$

2. $-I \notin S$. *The proof follows same as above.*

Both these conditions also turn out to be sufficient for code $C[S]$ to be non-empty.

4.3 Dimension of stabilizer code

To analyze properties of a stabilizer code, it will be useful to have a compact description of a group using its *generators*.

Definition 4.4. *A set of elements $g_1, g_2, \dots, g_l$ in a group S generate the group S if every element of S can be written as product of elements from the list $g_1, g_2, \dots, g_l$. We denote this as $S = \langle g_1, \dots, g_l \rangle$.*

Ideally, we want our generators $g_1, g_2, \dots, g_l$ to be independent in the sense that removing any generator g_i makes the group generated smaller.

Definition 4.5. $g_1, g_2, \dots, g_l$ are dependent if $\exists i$ such that g_i can be written as the product of g_j 's, $j \neq i$.

Suppose $C[S]$ has n physical qubits i.e is a subspace in $(\mathbb{C}^2)^{\otimes n}$. Then, $C[S]$ is non-empty provided S is generated by independent commuting generators and $-I \notin S$. Indeed, if there are l independent generators, then $C[S]$ is 2^{n-l} dimensional, based on an intuitive idea that each independent parity check i.e. projecting on $(I + g_i)/2$ decreases the dimension by a factor of $1/2$.

Recall we use the notation $[[n, k, d]]$ -QECC to denote a quantum error correcting code which has n physical qubits, k logical qubits and distance d .

Fact 4.6 (Dimension of Stabilizer Codes). *Suppose $g_1, \dots, g_l$ are independent. Let $S = \langle g_1, \dots, g_l \rangle$. Then, $C[S]$ is a $[[n, n-l, _]]$ QECC.*

Proof. **TODO: Write the proof from class.**

□

4.4 Distance of stabilizer code

Recall the distance of a code is defined in terms of the weight of errors that can be corrected by the code. But let's start with an easier question: when can stabilizer code $C[S]$ detect errors? Suppose $C[S]$ is corrupted by error E .

1. If error E anti-commutes with an element $P \in S$. Then, for any codestate $|\psi\rangle \in C[S]$, we get that $E|\psi\rangle$ is in the -1 eigenspace of P ,

$$PE|\psi\rangle = -EP|\psi\rangle = -E|\psi\rangle.$$

In this case, E takes $C[S]$ to an orthogonal subspace, and the error can be detected by performing the projective measurement associated to element P .

2. The real danger comes when E commutes with every $P \in S$. Then, the error is undetectable because $E|\psi\rangle$ also lives in the subspace $C[S]$,

$$PE|\psi\rangle = EP|\psi\rangle = E|\psi\rangle.$$

This motivates the following subgroup

Definition 4.7. *The centralizer of S in Pauli_n is the set of elements in Pauli_n which commutes with all elements*

$$Z(S) = \{E \in \text{Pauli}_n : EP = PE \text{ for all } P \in S\}$$

For stabilizer groups we are interested in, the centralizer is equal to its normalizer, denoted $N(S)$, which is defined to consist of all elements E of Pauli_n such that $EPE^\dagger \in S$ for all $P \in S$.

$$N(S) = \{E \in \text{Pauli}_n : EPE^\dagger \in S \text{ for all } P \in S\}$$

Fact 4.8. $N(S) = Z(S)$ for any subgroup S of Pauli_n not containing $-I$.

Proof. **TODO:** Finish this proof. It is easy to check that $Z(S) \subset N(S)$. To get the other direction, recall that $E \in N(S)$ either commutes or anti commutes with $P \in S$ since both belong to Pauli_n .

□

Continuing the discussion at the beginning of the section, if error $E \in N(S) - S$, then the error affects the states in $C[S]$ but we can not detect it.

Fact 4.9. The distance of stabilizer code $C[S]$ is the smallest weight in $N(S) - S$.

Proof. Let d be the smallest weight in $N(S) - S$. Then recall $C[S]$ has distance d iff $\langle \psi | E | \psi \rangle = 0$ for all weight $< d$ Pauli E .

Let E be some Pauli with weight $< d$. Then, since d is the smallest weight Pauli in $N(S) - S$, we get that $E \notin N(S) - S$. This leads to 2 cases:

1. $E \in S$. In this case, E stabilizes the codestate i.e. $E|\psi\rangle = |\psi\rangle$. Therefore,

$$\langle \psi | E | \psi \rangle = \langle \psi | \psi \rangle = 1$$

2. $E \notin N(S)$. In this case, there exists $P \in S$ which anti-commutes with E i.e. $PE = -EP$.

$$\begin{aligned} \langle \psi | E | \psi \rangle &= \langle \psi | EP | \psi \rangle = -\langle \psi | PE | \psi \rangle = -\langle \psi | E | \psi \rangle \\ &\implies \langle \psi | E | \psi \rangle = 0 \end{aligned}$$

□

We now give another proof which gives a bit more intuition on how to fix errors with weight $\leq (d-1)/2$. We begin with a definition.

Definition 4.10. We say two errors E_1 and E_2 are degenerate if

$$E_1 = E_2 P \text{ for some } P \in S$$

Note that this definition is symmetric as $P^2 = I$ (because P has to be an observable. Non-observable P 's square to $-I$, which would imply that the code $C[S]$ is empty.). A nice fact about degenerate errors is that fix for an error also works for its degenerate errors. Note that a fix for unitary error E_2 is its transpose $E_2^\dagger$.

$$E_1 |\psi\rangle \xrightarrow{\text{fix for } E_2} E_2^\dagger E_1 |\psi\rangle = E_2^\dagger E_2 P |\psi\rangle = P |\psi\rangle = |\psi\rangle$$

We now give another proof for distance of stabilizer code by proving the following fact.

Fact 4.11. *Let $d = 2t + 1$ be the smallest weight in $N(S) - S$. Suppose E_1, E_2 are Pauli errors with weight $\leq t$. Then, either*

1. E_1 and E_2 have different syndrome, or
2. E_1 and E_2 are degenerate errors.

Proof. **TODO: Finish proof from class.**

□

4.5 Logical operators

The normalizer $N(S)$ for a stabilizer group are also called logical operators.

Example 4.12. *To motivate this terminology, let's look at Shor's 9-qubit code, which is a stabilizer code.*

1. *The stabilizer for Shor code has eight generators: $Z_1Z_2, Z_2Z_3, Z_4Z_5, Z_5Z_6, Z_7Z_8, Z_8Z_9, X_1X_2X_3X_4X_5X_6, X_4X_5X_6X_7X_8X_9$.*

2. The codespace $C[S]$ is generated by the logical states

$$\begin{aligned} |0\rangle_L &= \left(\frac{|000\rangle + |111\rangle}{\sqrt{2}} \right)^{\otimes 3} \\ |1\rangle_L &= \left(\frac{|000\rangle - |111\rangle}{\sqrt{2}} \right)^{\otimes 3} \end{aligned}$$

Note that $S \subset N(S)$, and act like identity matrix on the codespace $C[S]$. The elements of $N(S) - S$ represent operations on the encoded logical qubit

1. $\bar{Z} = X_1 X_2 \dots X_9$ which acts like logical Z -gate i.e.

$$\begin{aligned} X_1 X_2 \dots X_9 |0\rangle_L &= |0\rangle_L \\ X_1 X_2 \dots X_9 |1\rangle_L &= -|1\rangle_L \end{aligned}$$

2. $\bar{X} = Z_1 Z_4 Z_7$ which acts like logical X -gate i.e.

$$\begin{aligned} Z_1 Z_4 Z_7 |0\rangle_L &= |1\rangle_L \\ Z_1 Z_4 Z_7 |1\rangle_L &= |0\rangle_L \end{aligned}$$

We define the equivalence class of distinct logical operations as $N(S)/S = \{E \cdot S : E \in N(S)\}$. Here $E \cdot S$ is an equivalence class of degenerate errors, given by the set $\{EP : P \in S\}$. We now claim (without proof) that the phenomenon above is true for all stabilizer codes.

Fact 4.13. Suppose $C[S]$ has n physical qubits and l independent commuting generators. Then, $N(S)/S$ is isomorphic to Pauli_{n-l} .

Chapter 5

Calderbank-Shor-Steane codes (Week 5)

In this lecture, we will discuss CSS codes, which are a subclass of stabilizer codes. But before we will briefly review classical linear codes.

5.1 Classical linear codes

Definition 5.1. A classical code is a subset $C \subset \mathbb{F}_2^n = \{0, 1\}^n$. It is linear if it forms a subspace: for all $x, y \in C$, $x + y \in C$.

This is equivalent to existence of linearly independent basis elements $g_1, g_2, \dots, g_k$ such that $C = \text{span}\{g_1, g_2, \dots, g_k\}$. Note that because we are in $\mathbb{F}_2$, this can be simplified to $\{x_1 g_1 + \dots + x_k g_k : (x_1, \dots, x_k) \in \{0, 1\}^k\}$.

To represent C in more familiar terms, we define the generator matrix.

Definition 5.2 (Generator matrix). The generator matrix G for a code with linearly independent basis elements $g_1, g_2, \dots, g_k$ is given by

$$G = \begin{bmatrix} | & | & | & | \\ g_1 & g_2 & \cdots & g_k \\ | & | & | & | \end{bmatrix}$$

A linear code can then be represented in terms of its generator matrix as

$$C = \{G \cdot x : x \in \{0, 1\}^k\}.$$

In this case, $\dim(C) = k$ as it encodes k bits, $x \in \{0, 1\}^k$ into n bits, $G \cdot x \in \mathbb{F}_2^n$.

A good example is classical repetition code $C = \{000, 111\}$ which is generated by $g_1 = 111$. The generator matrix is

$$G = \begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix}$$

And how do we check if a string is in my linear code? We perform a parity check. A parity check is an $h \in \{0, 1\}^n$ such that $\langle h, c \rangle = 0$ for all codewords $c \in C$. The set of parity checks also form a linear subspace.

Definition 5.3 (Dual code). *The dual code $C^\perp \subset \mathbb{F}_2^n$ is given by the set of all parity checks:*

$$C^\perp = \{h \in \{0, 1\}^n : \langle h, c \rangle = 0 \text{ for all codewords } c \in C\}$$

We can verify that if $\dim(C) = k$, then $\dim(C^\perp) = n - k$. Similar to generator matrix, we use the linearly independent parity checks to define the parity check matrix.

Definition 5.4 (Parity check matrix). *The parity check matrix H for code C with linear independent basis of parity checks $h_1, h_2, \dots, h_{n-k}$ is given by*

$$H = \begin{bmatrix} | & | & | & | \\ h_1 & h_2 & \cdots & h_{n-k} \\ | & | & | & | \end{bmatrix}$$

For example, the parity checks for repetition code are $h_1 = (1, 1, 0)$, $h_2 = (0, 1, 1)$ and $h_3 = (1, 0, 1)$. Here h_1, h_2 form a linearly independent basis for the parity checks. The parity check matrix is

$$H = \begin{bmatrix} 1 & 0 \\ 1 & 1 \\ 0 & 1 \end{bmatrix}$$

A nice fact about the dual code is that its dual is the original code.

Fact 5.5. $(C^\perp)^\perp = \{c \in \{0, 1\}^n : \langle h, c \rangle = 0 \forall \text{ parity checks } h \in C^\perp\} = C$

This means that the generator and parity check matrix for dual code $C^\perp$ are given by H and G . Also note that $H^\top G = 0$.

5.2 Distance of classical linear codes

How do we fix errors in linear codes? Suppose we encode message $x \in \mathbb{F}_2^k$ into n bits as $c = Gx$, but noise corrupts c into a corrupted codeword $c' = c + e$. Because $H^\top c = 0$, the parity checks on c' , given by

$$H^\top c' = H^\top c + H^\top e = H^\top e.$$

Just like quantum codes, $H^\top e$ is called the syndrome. But unlike quantum codes, to correct all errors $e \in \mathcal{E}$, we need to uniquely identify e . That is, for all $e_1, e_2 \in \mathcal{E}$, we need

$$H^\top e_1 \neq H^\top e_2.$$

Definition 5.6 (Distance). *The distance of linear code C is*

$$\begin{aligned} \min_{x \neq y \in C} \text{HammingDist}(x, y) &= \min_{x \neq y \in C} |x - y| \\ &= \min_{x \neq y \in C} |x + y| \\ &= \min_{x \in C, x \neq 0} |x| \end{aligned}$$

Fact 5.7. *The distance of linear code C is $2t+1 \implies C$ can correct $\leq t$ errors.*

Proof. **TODO: Finish proof from class.**

□

5.3 CSS code

CSS codes are stabilizer codes with generators which are (1) all X or (2) all Z . The main idea will be to use the parity checks from linear codes to build the stabilizers.

Given $h \in \{0, 1\}^n$, define $X^h = X^{h_1} \otimes \dots \otimes X^{h_n}$, where $X^0 = I$ and $X^1 = X$. We will use the notation $[n, k, d]$ to denote a classical linear error correcting code which encodes k bits onto n bits and has distance d .

Definition 5.8 (CSS code). *Let $C_X, C_Z \subset \mathbb{F}_2^n$ be linear error correcting codes. The corresponding CSS code is the stabilizer code with generators X^{h_x} and Z^{h_z} for all $h_x \in C_X^\perp$ and $h_z \in C_Z^\perp$.*

Fact 5.9. *To ensure CSS code is non-empty, we need that*

$$C_X^\perp \subset C_Z \text{ or equivalently } C_Z^\perp \subset C_X.$$

In other words, X parity checks are Z codewords, and Z parity checks are X codewords.

Proof. CSS codes are stabilizer codes, so for them to be non-empty, we need the generators to commute:

$$X^{h_x} Z^{h_z} = Z^{h_z} X^{h_x} \text{ for all } h_x \in C_X^\perp \text{ and } h_z \in C_Z^\perp.$$

But because Pauli matrices X and Z anti-commute, we can write

$$X^{h_x} Z^{h_z} = (-1)^{\langle h_x, h_z \rangle} Z^{h_z} X^{h_x},$$

and therefore the commutation requirement above is equivalent to C_X and C_Z parity checks being orthogonal

$$\langle h_x, h_z \rangle = 0 \pmod{2} \text{ for all } h_x \in C_X^\perp \text{ and } h_z \in C_Z^\perp.$$

This also means that a fixed $h_x \in C_X^\perp$ satisfies $\langle h_x, h_z \rangle$ for all $h_z \in C_Z^\perp$ i.e. $h_x \in (C_Z^\perp)^\perp$. Noting that a code is its dual's dual completes the proof. □

Fact 5.10. Let C_X, C_Z be $[n, k_x, d_x]$ and $[n, k_z, d_z]$ linear ECC respectively. The corresponding CSS code is a $[[n, k_x + k_z - n]]$ code, assuming $k_x + k_z > n$.

Proof. The number of independent parity checks in C_X, C_Z are $n - k_x$ and $n - k_z$ respectively. Therefore, the number of parity checks for CSS code are $2n - k_x - k_z$, which gives the logical qubits $n - (2n - k_x - k_z) = k_x + k_z - n$ $\square$

5.4 Distance of CSS code

Since, CSS code is a stabilizer code, we can compute its normalizer to find its distance.

Fact 5.11. The distance of CSS code is $\min \{d_x^+, d_z^+\}$ where

$$d_Z^+ = \min_{e \in C_Z - C_X^\perp} |e| \geq \min_{e \in C_Z - \{0\}} |e| = d_z$$

$$d_x^+ = \min_{e \in C_X - C_Z^\perp} |e| \geq \min_{e \in C_X - \{0\}} |e| = d_x$$

To build some intuition for this, let's look at what errors can be detected by a CSS code¹?

Let's start with bit flip errors. For $e \in \mathbb{F}_2^n$, consider the error X^e . Recall that X^e can be detected by our stabilizer code if and only if it anti commutes with one of the stabilizers. Since, X^e commutes with every X -generator, by Section 4.4, it can not be detected by X -type generators. In other words, X errors are supposed to be detected by Z -type generators. We claim X^e is detectable by Z -type generators if and only if $e \notin C_Z^\perp$. This is reminiscent of classical codes where we can detect errors as long as they do not belong to the code.

But unlike classical codes, not all errors X^e for $e \in C_Z$ need to be detected/corrected. By our definition of CSS code, $C_X^\perp$ is a subset of C_Z . And even though by above discussion, we cannot detect errors X^e for $e \in C_X^\perp$, these errors are benign as X^e is a stabilizer for our CSS code. Together, this means that the only non-benign undetectable errors are in $C_Z - C_X^\perp$. We now prove this more formally.

Proof. **TODO: Finish proof from class.**

$\square$

¹since weight of errors corrected is usually half the errors detected, this is a reasonable choice.

²This is because for any Z parity check $h_z \in C_Z^\perp$, the error X^e anti-commutes with a Z -type generator $Z^{h_z} \iff \langle e, h_z \rangle \neq 0$ as

$$X^e Z^{h_z} = (-1)^{\langle e, h_z \rangle} Z^{h_z} X^e.$$

Chapter 6

Toric Code (Week 6)

In this lecture, we study Toric code which was introduced by Kitaev in 1997. The toric code is defined on an $L \times L$ grid on a torus with the left most edge is identified with the right most edge, and the upper edge is identified with the lower edge. This is equivalent to 2-D grid with periodic boundary conditions.

TODO: Draw a L by L toric code (for $L = 4$) showing vertices, edges and plaquettes highlighting qubits which are “repeated”. Hint: this will be $L+1$ by $L+1$ grid with repeated edges

On an $L \times L$ grid, there are L^2 vertices, $2L^2$ edges (two edges associated to each vertex), and L^2 plaquettes (1×1 squares, one square associated to each vertex). We associate a qubit to each edge of this grid. A toric code therefore has $n = 2L^2$ physical qubits.

6.1 Stars and plaquettes

Toric code is a CSS code, so its stabilizers are given by parity checks of two classical linear codes C_X and C_Z .

- The parity checks for C_X are generated by the following parity checks: for every vertex x , parity check h_x is 1 on all edges incident to the vertex, and 0 everywhere else.
- The parity checks for C_Z are generated by the following parity checks: for every plaquette z , parity check h_z is 1 on all edges which are part of the plaquette, and 0 everywhere else.

TODO: Draw parity checks for C_X and C_Z codes.

For this code to be non-empty, again by Fact 4.3, the stabilizers of the CSS code must commute, which turns out to be the case.

Fact 6.1. For each C_Z parity check h_z and C_X parity check h_x ,

$$X^{h_x} Z^{h_z} = Z^{h_z} X^{h_x}$$

Proof. This is because $\langle h_x, h_z \rangle = 0$ and

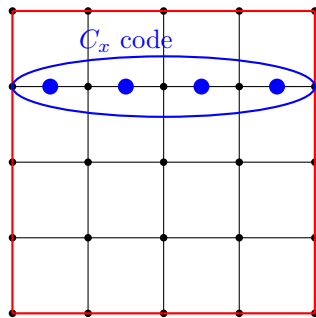
$$X^{h_x} Z^{h_z} = (-1)^{\langle h_x, h_z \rangle} Z^{h_z} X^{h_x}.$$

If the corresponding plaquette p and vertex p do not overlap, then $\langle h_x, h_z \rangle = 0$. Otherwise, vertex v is a corner of plaquette p , and they overlap on 2 edges, in which case again $\langle h_x, h_z \rangle = 0$. $\square$

6.2 Closed loop and boundary

Since Toric code is a CSS code, its parameters can be inferred from the properties of its base classical codes C_X, C_Z , and their duals $C_X^\perp, C_Z^\perp$. Because of the underlying symmetries, we would only need to analyze C_X and $C_Z^\perp \subset C_X$.

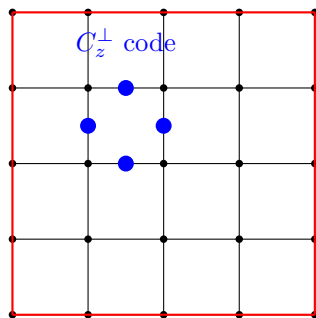
TODO: Draw an element of C_X code.



Fact 6.2. $C_X = \text{span} \{ \mathbb{1}_x : x \text{ is a closed loop on the edges} \}$.

Proof. We can verify this as follows. The parity checks for C_X code are associated to vertices: each vertex adds up the bits on the edges incident on it, and checks if it is 0. We can check all these checks are satisfied for any closed loop on the edges. Now suppose you have an assignment of 0 and 1's to edges which is not in the span of closed loop i.e. has a vertex with only one excited edge or 3 excited edges. Such a vertex's parity checks are not satisfied. $\square$

TODO: Draw an element of $C_Z^\perp$ code.



As expected, the parity checks for C_Z code are a subset of C_X .

Fact 6.3. $C_Z^\perp = \text{span} \{ \mathbb{1}_x : x \text{ is a boundary of set of plaquettes} \}$.

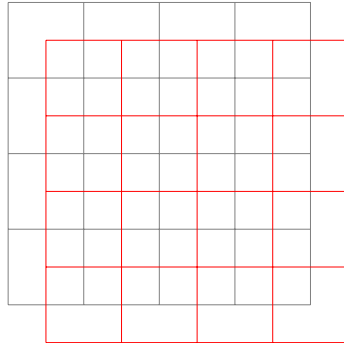
Proof. This can be verified by noticing that $C_Z^\perp$ is the subspace spanned the parity checks associated to plaquettes: each plaquette adds up the bits on the edges which are part of it, and checks if it is 0. Suppose we sum two such parity checks. If the corresponding plaquettes do not overlap, then their sum is again a boundary of a set of plaquettes. Otherwise, they intersect on a single edge, and form the boundary of a larger plaquette. $\square$

To understand the C_Z and $C_X^\perp$ code, we look at the dual grid, by shifting the primal grid to half a cell down and right. All the vertices and plaquettes of the primal grid, take the roles of plaquettes and vertices in the dual grid respectively.

Fact 6.4. $C_Z = \text{span} \{ \mathbb{1}_x : x \text{ is a closed loop on the edges in the dual grid} \}$.

Fact 6.5. $C_X^\perp = \text{span} \{ \mathbb{1}_x : x \text{ is a boundary of set of plaquettes in the dual grid} \}$.

TODO: Draw the dual and primal grid.



Primal Grid (black) and Dual Grid (red)

We can use this description of code C_X and $C_X^\perp$ to compute its parameters.

Fact 6.6. C_X, C_Z are both $[2L^2, L^2 + 1, 4]$ error correcting codes.

Proof. As discussed before, the number of physical qubits are $2L^2$. We compute the dimension and distance of these codes:

1. The parity checks $C_X^\perp$ is spanned by boundaries of individual plaquettes, a set of size L^2 . But these do not form an independent set, as the boundaries of all the individual plaquettes sum up to 0. But any subset formed by removing any one of the plaquette boundaries is an independent set of size $L^2 - 1$.
2. The distance is equal to the minimum weight of element of C_X . Since, the smallest closed loop is the one around a plaquette, the distance of C_X code is 4.

□

6.3 Distance of toric code

Using the machinery of CSS codes, and Fact 6.6, we get that toric code is a $[[2L^2, 2, d^+]]$ QECC where

$$d^+ = \min \{d_Z^+, d_X^+\} \geq \min \{d_X, d_Z\} = 4$$

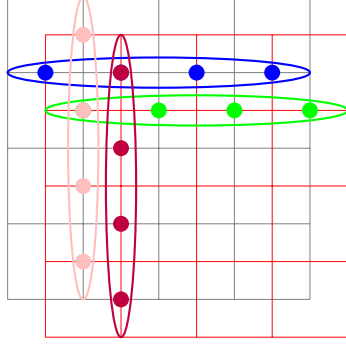
How big is d^+ compared to d_X or d_Z ? Recall

$$d_X^+ = \min_{x \in C_X - C_Z^\perp} |x|$$

where C_X is the set of all closed loops, and $C_Z^\perp$ is the set of all closed which are boundaries of plaquette sets.

What are the closed loops which are not a boundary of a plaquette set? These are the loops which winds around the torus. The smallest such loop is of length L .

TODO: Draw the 4 canonical closed loops which are not boundaries of a plaquette set.



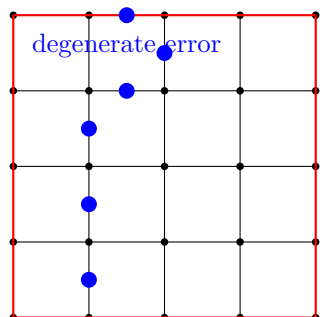
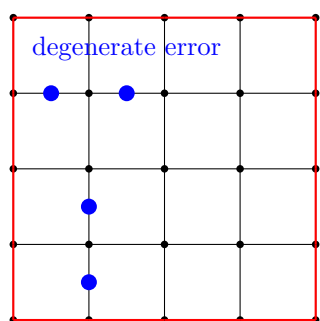
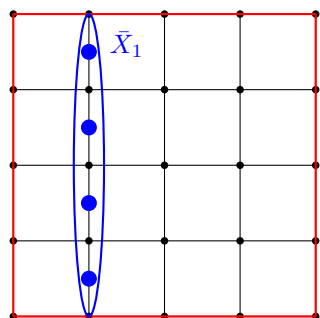
Fact 6.7. *Toric code is a $[[2L^2, 2, L]]$ QECC.*

The large difference between d_X^+ and d_X is because all the small weight errors are degenerate errors, and in the equivalence class of no error/identity logical operator.

Recall the equivalence class of distinct logical operations for a stabilizer code $C[S]$ is $N(S)/S = \{E \cdot S : E \in N(S)\}$. Here S is the stabilizer group, and $E \cdot S$ is an equivalence class of degenerate errors, given by the set $\{EP : P \in S\}$. Note that two errors are degenerate if and only if they can be transformed into one another by applying a stabilizer $P \in S$.

For toric code, the equivalence classes of degenerate errors (also called logical operators) are characterized by 4 loops: top-bottom X loop ($\bar{X}_1$), left-right X loop ($\bar{X}_2$), top-bottom Z loop ($\bar{Z}_1$) and left-right Z loop ($\bar{Z}_2$). We can verify how applying stabilizers of toric code, plaquette parity checks and star parity checks, lead to topologically equivalent loops.

TODO: Draw $\bar{X}_1$ and 2 examples of errors degenerate to $\bar{X}_1$.



We will see in next few lectures, how toric code is a quantum generalization of repetition code.

Chapter 7

Chain Complexes (Week 7)

In this lecture, we will introduce a useful formalism from algebraic topology, Chain Complexes, which will help us in studying quantum codes. We will see that classical linear codes are 1-dimensional chain complex, and quantum codes, which are generalizations of classical codes, are higher dimensional chain complexes. This perspective will help us in designing the “good” quantum LDPC codes from classical codes.

7.1 Chain Complexes

We start by defining chain complexes. For the rest of the course, we will only consider vector spaces over $\mathbb{F}_2$.

Definition 7.1 (Chain Complexes). *A d -dimensional chain complex $C = (C_0, C_1, \dots, C_d)$ is a sequence of vector spaces, called i -chain spaces, C_i equipped with linear maps, called i -boundary maps,*

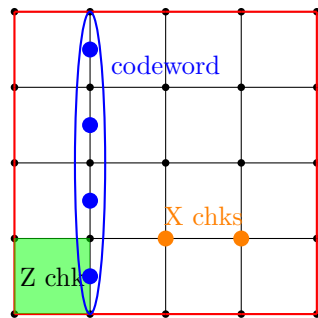
$$\partial_i: C_i \rightarrow C_{i-1}$$

such that the boundary of the boundary is identically zero:

$$\partial_i \circ \partial_{i+1} = 0.$$

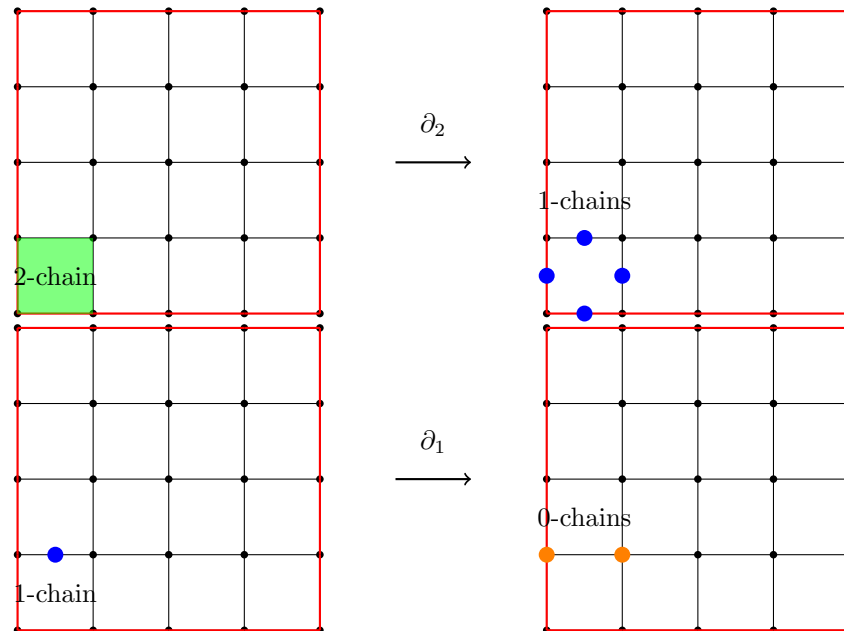
Elements in C_i are called i -chains, and we will think of an i -chain as an i -dimensional object. As an example, let's look at the chain complex associated with toric code. Recall toric code is defined on an $L \times L$ grid on a torus. Let V be the vertices, E be the edges, and F be the faces/plaquettes of the grid. The 2-dimensional chain complex for toric code is $C = (C_0, C_1, C_2)$ with $C_0 = \mathbb{F}_2^V$, $C_1 = \mathbb{F}_2^E$, and $C_2 = \mathbb{F}_2^F$. In other words, a 0-chain is an element $v \in \mathbb{F}_2^V$, a 1-chain is an element $e \in \mathbb{F}_2^E$, and a 2-chain is an element $f \in \mathbb{F}_2^F$.

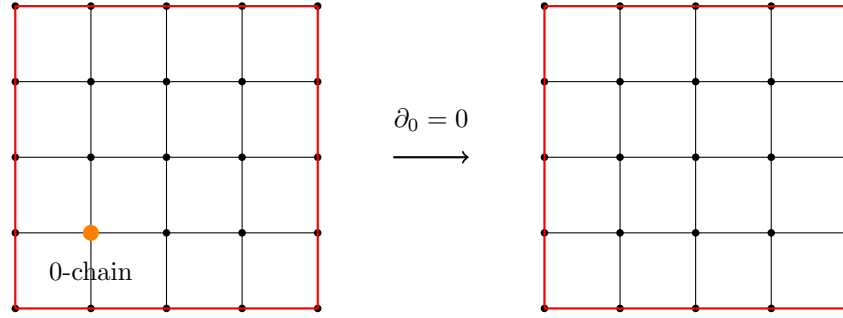
TODO: Draw Z checks as 2-chain, codewords as 1-chain, and X checks as 0-chain.



The boundary maps ∂_2, ∂_1 map a 2-chain to its 1-chain boundary, and a 1-chain to its 0-chain boundary respectively.

TODO: Draw examples on how the boundary maps act on a 2-chain, 1-chain, 0-chain. Observe how the $(\partial_1 \circ \partial_2)(f)$ for any face f is zero. Observation: from the second figure below, ∂_1 maps an 1-chain, i.e. edge, to its two endpoints of the edge. Since we have 4 edges in a loop, four pairs of the endpoints cancel out each other and derive 0. So $(\partial_1 \circ \partial_2)(f)$ for any face is zero.





More formally, we define ∂_2 for an indicator of face f as the edge indicator of f 's boundary. Then, we extend it by linearity to all 2-chains. We abuse notation and use ∂_2 to denote both the boundary map, and its matrix representation. ∂_2 is an $|E| \times |F|$ matrix, with its f th column given by the edge boundary of f .

$$[\partial_2]_{e,f} = \begin{cases} 1 & \text{if edge } e \text{ borders face } f \\ 0 & \text{otherwise} \end{cases}$$

This is exactly the parity check matrix H_Z for Z -code C_Z . Recall the plaquette parity check for a face f adds up the bits on its edge boundary, and checks if it is 0. Similarly, the matrix ∂_1 is a $|V| \times |E|$ matrix, with its e th column given by its vertex endpoints.

$$[\partial_1]_{v,e} = \begin{cases} 1 & \text{if vertex } v \text{ is an endpoint of edge } e \\ 0 & \text{otherwise} \end{cases}$$

And this is transpose of the parity check matrix H_X for X -code C_X . Recall the star parity check for a vertex v adds up the bits on the edges incident to it, and checks if it is 0.

Because the parity checks H_X and H_Z are orthogonal (for CSS code to be non-empty), we get that the boundary of a boundary is zero:

$$\partial_1 \partial_2 = H_X^\top H_Z = 0.$$

This discussion can be easily generalized to any CSS code. A CSS code with parity check matrices H_X and H_Z is a chain complex:

$$C_2 \xrightarrow{H_Z} C_1 \xrightarrow{H_X^\top} C_0$$

which gives us the following fact:

Fact 7.2. *CSS codes are 2-dimensional chain complexes.*

We note here that classical linear codes are 1-dimensional chain complexes. Later on, we will see a general method to build higher dimensional chain complexes from lower dimensional chain complexes. And use this to show how toric codes are quantum generalizations of classical repetition codes! Recall in second lecture, we also derived Shor's code also as a generalization of repetition code.

7.2 Homology of a chain complex

We now show how the different parameters of a quantum code can be inferred from the homology of the associated chain complex. We first define the analogs of boundaries and closed loops we saw in toric codes.

Definition 7.3 (*i*-cycles and *i*-boundaries). *For a d -dimensional chain complex $C = (C_0, \dots, C_d)$, we define *i*-cycles $Z_i(C)$, *i*-boundaries $B_i(C)$ as follows:*

- *We say an i -chain e is an i -cycle if it has no boundary i.e. $\partial_i(e) = 0$. We denote by $Z_i(C)$, the set of all i -cycles*

$$Z_i(C) = \ker \partial_i \subset C_i$$

- *We say an i -chain e is an i -boundary (of some higher dimensional element f in C_{i+1}) if it is of form $\partial_{i+1}(f)$ for some $f \in C_{i+1}$. We denote by $B_i(C)$, the set of all i -boundaries*

$$B_i(C) = \text{im } \partial_{i+1} \subset C_i$$

Now because boundary of a boundary is zero: $\partial_i \circ \partial_{i+1} = 0$, we get that $\partial_i B_i(C) = 0$. In other words, $B_i(C) \subset Z_i(C)$. Since, both $B_i(C)$ and $Z_i(C)$ are subspaces of C_i , we say i -cycles $e, e' \in Z_i(C)$ are homologically equivalent if $e = e' + b$ for some $b \in B_i(C)$.

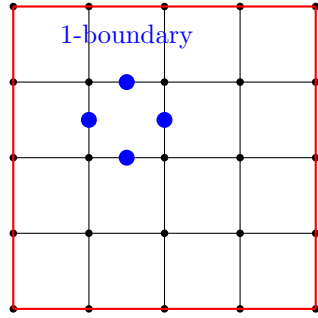
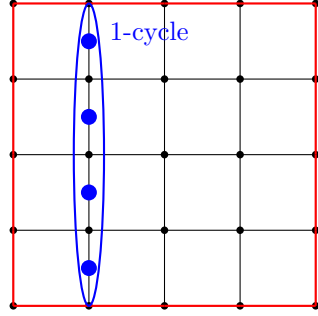
Definition 7.4. *The i th homology group $H_i(C)$ is the equivalence class of i -cycles:*

$$H_i(C) = Z_i(C)/B_i(C)$$

We continue with our example of 2-dimensional chain complex $C = (C_0, C_1, C_2)$ for toric code. The 1-cycles $Z_1(C)$ are exactly the elements of X-code C_X , and the 1-boundaries $B_1(C)$ are exactly the elements of parity checks $C_Z^\perp$. In particular, recall that 1-chains C_1 are $\mathbb{F}_2$ labellings of edges. Consider a Z-error Z^e for a 1-chain e , where the chain e specifies which qubits/edges have the Z error. Then,

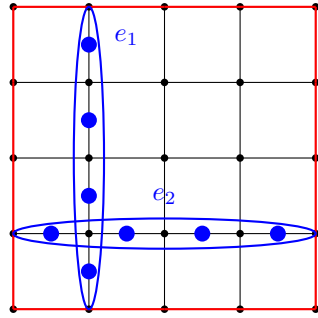
- the 1-chain e , such that Z^e is the top-bottom loop $\bar{Z}_1$, is a 1-cycle in $Z_1(C)$,
- the 1-chain e , such that Z^e is on the boundary of a plaquette set, is a 1-boundary in $B_1(C)$.

TODO: Draw examples of 1-boundaries and 1-cycles of toric code.



Furthermore, errors $Z^e, Z^{e'}$ are degenerate if $e, e' \in Z_1(C)$ belong to the same equivalence class in $H_1(C) = Z_1(C)/B_1(C)$. In fact, we can write the homology group $H_1(C)$ explicitly as $H_1(C) = \{0, e_1, e_2, e_1 + e_2\}$ where Z^{e_1}, Z^{e_2} are the top-bottom and left-right loop errors.

TODO: Draw the errors corresponding to the homology group.



Observe that similar to toric code, in general, the number of physical qubits $n = \dim C_1$ and the distance d_Z^+ is the minimum weight of a representative of a non-trivial element of $H_1(C)$.

7.3 Dual chain

For toric code, we studied the X -parity checks by looking at the dual grid. We now discuss the analog of this phenomenon in chain complexes.

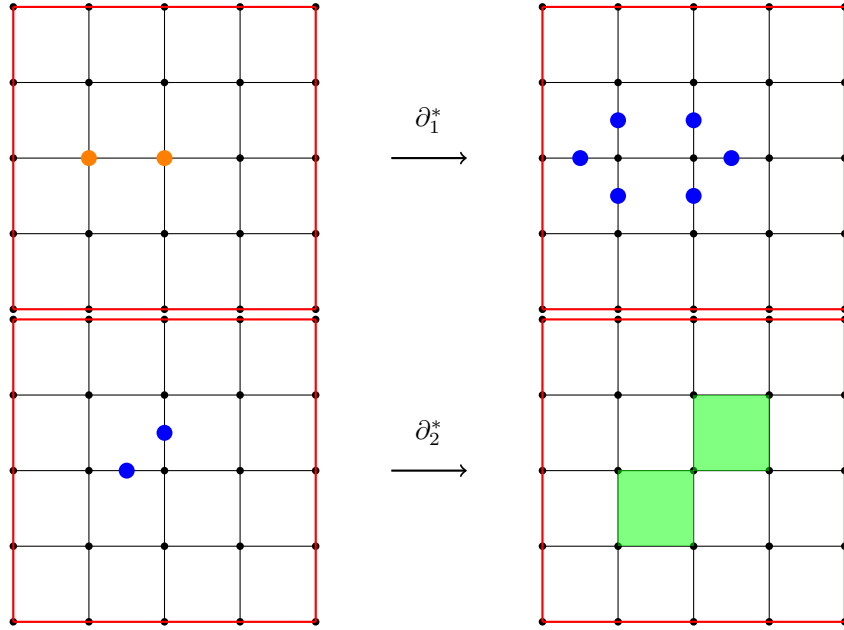
Definition 7.5 (Dual chain complex). For a d -dimensional chain complex $C = (C_0, \dots, C_d)$ with boundary maps ∂_i , consider the adjoint operators¹ ∂_i^*

$$\partial_{i+1}^* : C_i^* \rightarrow C_{i+1}^*$$

The vector spaces C_i^* are called i -cochain spaces and the vectors of C_i^* are called i -cobochains. And the coboundary of a coboundary is zero:

$$\partial_{i+1}^* \circ \partial_i^* = 0$$

TODO: Consider a 0-cobochain, a labelling of vertices on a 2D grid. Apply its boundary map $\partial_1^* : C_0^* \rightarrow C_1^*$, which sends it to its coboundary, the set of edges that the vertex parity checks involve. Equivalently, it is the set of edges incident to odd number of vertices in the 0-cobochain. Similarly, applying $\partial_2^* : C_1^* \rightarrow C_2^*$, the coboundary of 1-cobochain is the set of faces that are incident to an odd number of edges. Draw an example of this!



For toric code, the coboundary maps $\partial_1^* = \partial_1^\top = H_X$, and $\partial_2^* = \partial_2^\top = H_Z^\top$. We can similarly define coboundaries $B_i^* = \text{im}(\partial_i^*)$, cocycles $Z_i^* = \ker(\partial_{i+1}^*)$, cohomology groups $H_i^* = Z_i^* / B_i^*$. And as before, the distance d_X^+ is the minimum weight of a representative of a non-trivial element of $H_1^*(C)$. Moreover,

Fact 7.6. $B_i^* = Z_i^{\perp}$ and $Z_i^* = B_i^{\perp}$.

¹for a linear map $A : H_1 \rightarrow H_2$ between two Hilbert spaces, the adjoint operator is the linear operator $A^* : H_2 \rightarrow H_1$ satisfying $\langle Ah_1, h_2 \rangle_{H_2} = \langle h_1, A^*h_2 \rangle_{H_1}$. For our purposes, A^* is always $A^\top$.

Proof. Here is an informal proof.

$$B_i^* = \text{im } \partial_i^* = \{\partial_i^\top y\} \quad Z_i = \ker \partial_i = \{x : \partial_i x = 0\}$$

and for x, y above,

$$x^\top \partial_i^\top y = 0.$$

□

Together, this gives us the following relationship between the homology of a 2-dimensional chain complex (C_0, C_1, C_2) , and its corresponding CSS code where the X-type stabilizers are given by $\{X^c : c \in B_1^*\}$ and Z-type stabilizers are given by $\{Z^c : c \in B_1\}$. Recall $B_1^* = \text{im } \partial_1^*$ and $B_1 = \text{im } \partial_2$.

Fact 7.7. *Consider a d -dimensional chain complex $C = (C_0, C_1, \dots, C_d)$ with boundary maps $\partial_1, \dots, \partial_d$ which satisfy $\partial_i \circ \partial_{i+1} = 0$. Then, any (C_{i-1}, C_i, C_{i+1}) defines a $[[\dim C_i, \dim H_i(C), d^+]]$ CSS code where d^+ is the minimum weight of a representative of a non-trivial element of $H_i(C)$ or $H_i^*(C)$.*

Proof. To compute the dimension, note that:

$$\begin{aligned} k &= \dim(C_i) - \dim(B_i^*) - \dim(B_i) \\ &= \dim(C_i) - \dim(Z_i^\perp) - \dim(B_i) \\ &= \dim(Z_i) - \dim(B_i) \\ &= \dim(H_i^*) \end{aligned}$$

The distance follows from the discussion before.

□

Chapter 8

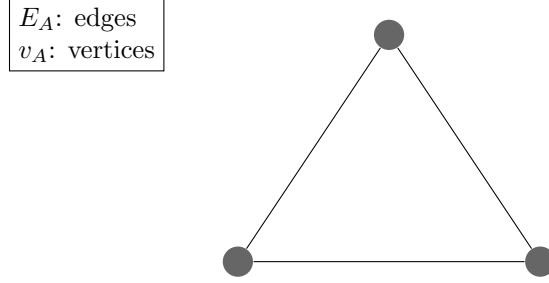
Hypergraph Product Codes (Week 8)

In this lecture, we will see a natural way to obtain a 2-dimensional chain complex from two 2-dimensional chain complexes. Since, 1-dimensional chain complexes are classical error correcting codes and 2-dimensional complexes are CSS codes, this gives a way to construct CSS codes from any two classical codes.

8.1 Classical codes as chain complexes

Let's start by studying a classical code as 1-dimensional chain complex $A = (A_0, A_1)$. Recall that A_1 contains 1-dimensional objects: edges E_A , and A_0 contains 0-dimensional objects: vertices V_A . Suppose we design a classical code C_A which assign variables to the 1-dimensional edges, and parity checks to 0-dimensional vertices.

TODO: Draw chain complex corresponding to classical code.



$$A_1 \xrightarrow{\partial_1} A_0$$

To compute parameters of this code, let's look at its homology.

- 1-cycles $Z_1(A)$ are the set of all codewords of C_A .

$$\begin{aligned} Z_1 &= \ker \partial_1 \\ &= \left\{ e \in \mathbb{F}_2^{|E_A|} : \partial_1(e) = 0 \right\} \\ &= C_A \end{aligned}$$

- 1-boundaries $B_1(A) = \{0\}$.

$$B_1(A) = \text{im } \partial_2 = \{0\}$$

- 0-cycles $Z_0(A) = A_0$ is the set of all 0-chains.

$$\begin{aligned} Z_0 &= \ker \partial_0 \\ &= \left\{ v \in \mathbb{F}_2^{|V_A|} : \partial_0(v) = 0 \right\} \\ &= \text{all 0-chains} \end{aligned}$$

- 0-boundaries $B_0(A)$ is the set of all possible syndromes, and $\dim(B_0) = \text{rank}(\partial_1)$.

$$\begin{aligned} B_0(A) &= \text{im } \partial_1 \\ &= \left\{ \partial_1(e) : e \in \mathbb{F}_2^{|E_A|} \right\} \\ &= \text{all possible syndromes} \end{aligned}$$

Together, these give us the dimensions of the homology groups for classical code C_A .

Fact 8.1. $\dim(H_1) = \dim(C_A)$ and $\dim(H_0) = \dim(C_A^\top)$ where $C_A^\top$ is the transpose code¹ to C_A .

Proof. TODO: The 1-homology group $H_1 = Z_1/B_1 = C_A$. Similarly, the 0-homology group $H_0 = Z_0/B_0$ has dimension $\dim(Z_0) - \dim(B_0) = |V_A| - \text{rank}(\partial_1) = \dim(C_A^\top)$. I proved above that $\dim(H_0) = \dim(C_A^\top)$. Add the proof for stronger statement, i.e. $H_0 = C_A^\top$.

□

Proof. By the first isomorphism theorem, we have:

$$C_0 / \text{im}(\delta_1) \cong \ker(\delta_1^T)$$

Denote the parity check matrix of code C_A as H , we have the following:

$$H_0 = C_0 / \text{im}(\delta_1) \cong \ker(H^T) = C_A^T$$

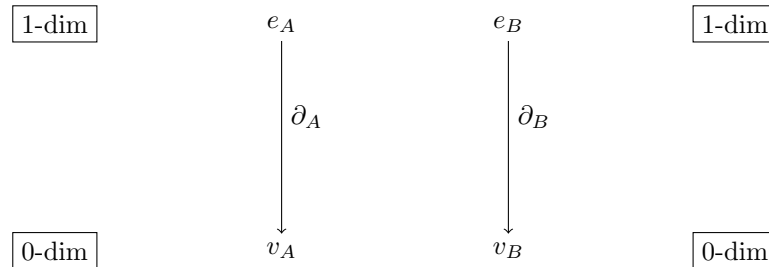
Here, the first equality comes from the fact that $Z_0 = \ker(\delta_0) = C_0$, and the last equality comes from the definition of transpose code. □

8.2 Hypergraph product code

Let $(A_0, A_1), (B_0, B_1)$ be two 1-dimensional chain complexes with boundary map ∂_A, ∂_B respectively. Here A_0 contains 0-dimensional objects (vertices) denoted v_A and A_1 contains 1-dimensional objects (edges) denoted e_A . Similarly, B_0 contains 0-dimensional objects (vertices) denoted v_B and B_1 contains 1-dimensional objects (edges) denoted v_B . The adjacency relationship between the vertices and edges is given by the boundary maps ∂_A and ∂_B .

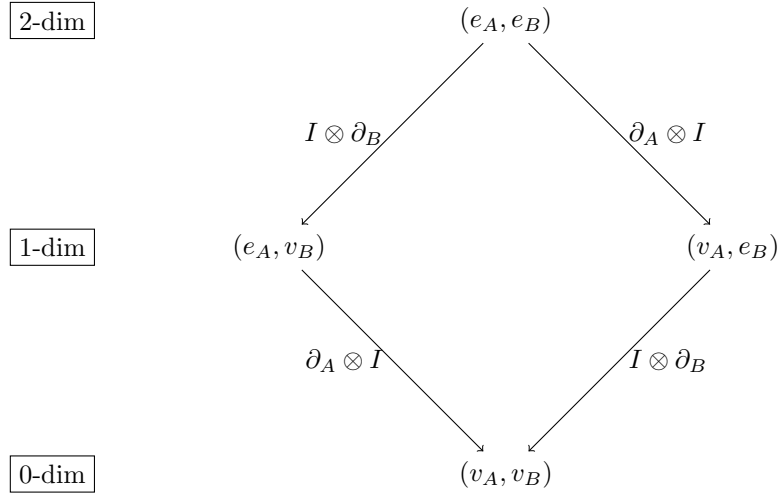
TODO: Draw the adjacency relationship of the base complexes.

¹For a classical code C with parity check matrix H , the transpose code $C^\top$ is the classical code with parity check matrix $H^\top$.



Before we define the tensor product of these chain complexes formally, we go over their structure informally. Their tensor product chain complex is a 3-dimensional chain complex $C = (C_0, C_1, C_2)$. Here C_0 contains 0-dimensional objects (vertices) (v_A, v_B) : tensor product of vertices, C_1 contains 1-dimensional objects (edges) (v_A, e_B) and (e_A, v_B) : tensor product of edges and vertices, and C_2 contains 2-dimensional objects (faces) (e_A, e_B) : tensor product of edges.

TODO: Draw the adjacency relationship of the tensor product complex.



We now define the tensor product of chain complexes formally.

Definition 8.2 (Tensor product of chain complexes). *Let $(A_0, A_1), (B_0, B_1)$ be two 1-dimensional chain complexes with boundary map ∂_A, ∂_B respectively. Then, their tensor product chain complex is a 2-dimensional chain complex $C = (C_0 = A_0 \otimes B_0, C_1 = A_0 \otimes B_1 \oplus A_1 \otimes B_0, C_2 = A_1 \otimes B_1)$ with boundary maps*

$$\partial_2 = \begin{bmatrix} \partial_A \otimes I \\ I \otimes \partial_B \end{bmatrix} \quad \text{and} \quad \partial_1 = \begin{bmatrix} I \otimes \partial_B & \partial_A \otimes I \end{bmatrix}$$

satisfying $\partial_1 \circ \partial_2 = 0$.

As discussed in last lecture, there is a natural correspondence between 2-dimensional chain complexes, and CSS codes. Given a chain complex $C = (C_0, C_1, C_2)^2$, we can construct a CSS code by assigning qubits to 1-dimensional objects (edges) in C_1 , Z checks to 2-dimensional objects (faces) in C_2 , and X checks to 0-dimensional objects (vertices) in C_0 . Therefore, the tensor product of 1-dim chain complexes defines a CSS code, which we call the hypergraph product code!

Theorem 8.3 (Distance of hypergraph product code). *Suppose, the base classical codes are: $C_A = [n_A, k_A, d_A]$, $C_B = [n_B, k_B, d_B]$, $C_A^\top = [n_A^\top, k_A^\top, d_A^\top]$, $C_B^\top = [n_B^\top, k_B^\top, d_B^\top]$. Then, the corresponding hypergraph product code is a $[[n, k, d]]$ CSS code with $n = n_A n_B^\top + n_A^\top n_B$, $k = \dim C_A \cdot \dim C_B^\top + \dim C_A^\top \cdot \dim C_B$, and $d \geq \min \{d_A, d_B, d_A^\top, d_B^\top\}$.*

Proof Sketch. Recall the parameters of a CSS code corresponding to a chain complex C_0, C_1, C_2 are: number of physical qubits $n = \dim C_1$, logical qubits $k = \dim H_1(C)$, and the distance d^+ is the minimum weight of a representative of a non-trivial element of $H_1(C)$.

²or any 3 consecutive layers of a chain complex

The dimension of 1-chain space $\dim C_1 = n_A n_B^\top + n_A^\top n_B$. To compute the number of logical bits, we can use Kunneth's formula, which writes the 1st homology group $H_1(C)$ in terms of homology groups of A and B complex.

$$H_1(C) = (H_1(A) \otimes H_0(B)) \oplus (H_0(A) \otimes H_1(B))$$

Using the following for classical codes,

$$\dim(H_1(A)) = \dim(C_A) \text{ and } \dim(H_0(A)) = \dim(C_A^\top).$$

we get that

$$\dim H_1(C) = \dim C_A \cdot \dim C_B^\top + \dim C_A^\top \cdot \dim C_B$$

The distance was shown to be $d^+ \geq \min \{d_A, d_B, d_A^\top, d_B^\top\}$ in Lemma 16 of [2]. $\square$

How do we pick the base classical codes? Suppose X be a $[n, k, d]$ classical code with $k, d = \Omega(n)$. To define the complex, we pick the independent parity checks $P_1, \dots, P_l$ with $l = n - k$. For this, the transpose code $X^\top$ is a $[l, 0, \infty]$ classical code. Now, if we take the base codes $C_A = X$ and $C_B = X^\top$, we get a $[[n^2 + l^2, \Omega(n^2), \Omega(n)]]$ code.

Corollary 8.4. *There exist hypergraph product codes which are $[[n, \Omega(n), \Omega(\sqrt{n})]]$ CSS codes.*

Compared to toric code, we kept the distance $\sqrt{n}$, and increased the logical qubits to n .

Chapter 9

Tanner Codes (Week 8)

We now introduce classical codes based on bipartite graphs. Here every vertex has a local view of a global code, which is required to be from a base code.

TODO: Draw a bipartite graph for tanner code.

Definition 9.1. Let $G = (L, R, E)$ be an r -regular bipartite graph. Let $C_0 \subset \mathbb{F}_2^r$ be a “base” code. Then, the corresponding Tanner code $\text{TC}(G, C_0)$ is defined as

$$\text{TC}(G, C_0) = \left\{ c \in \mathbb{F}_2^E : c|_{E(v)} \in C_0 \text{ for all } v \in L \cup R \right\}$$

where $E(v)$ are the edges incident on vertex v .

Throughout the course, we have been working with linear codes. So let's first show that if the base code is linear, then the corresponding Tanner code is a linear code.

Fact 9.2 (Tanner code is a linear code). If C_0 is linear, then $\text{TC}(G, C_0)$ is linear.

Proof. Let $c, c' \in \text{TC}(G, C_0)$. Then,

$$(c + c')|_{E(v)} = c|_{E(v)} + c'|_{E(v)} \in C_0$$

□

Fact 9.3 (Dimension of Tanner Code). *If G has m edges, n vertices and is r -regular, then*

$$\dim(\text{TC}(G, C_0)) \geq m - 2n(r - \dim(C_0))$$

Proof. C_0 has $r - \dim(C_0)$ many independent parity checks. Since, G has $2n$ vertices (each with $r - \dim(C_0)$ independent parity checks), $\text{TC}(G, C_0)$ has at most $2n(r - \dim(C_0))$ independent parity checks. □

What above fact really says is the following: if the base code has a constant rate, $\dim(C_0) = R_0 r$, then the corresponding Tanner code will also have constant rate, $\dim(\text{TC}(G, C_0)) = \Omega(m)$, as long as

$$\begin{aligned} m - 2n(r - \dim(C_0)) &= nr - 2n(r - R_0 r) \\ &= 2R_0 nr - nr \\ &= nr(2R_0 - 1) = m(2R_0 - 1) \end{aligned}$$

which means, we need $R_0 \in (1/2, 1]$.

9.1 Distance of Tanner Codes

To ensure tanner code has nice distance, we need the underlying graph G to be random-like.

Definition 9.4. A graph $G = (L, R, E)$ is ε -pseudorandom if for all $S \subset L$, $T \subset R$, we have

$$\left| E(S, T) - \frac{r}{n} |S| |T| \right| \leq \varepsilon r \sqrt{|S| |T|}$$

Fact 9.5 (Distance of Tanner Graph). *If G is ε -pseudorandom and distance of the base code $\text{dist}(C_0) = \Delta_0 r$, then the distance of tanner code*

$$\text{dist}(\text{TC}(G, C_0)) \geq \Delta_0(\Delta_0 - \varepsilon)m$$

Proof. **TODO:** Consider a $c \in \text{TC}(G, C_0)$. Define

$$\begin{aligned} S &= \left\{ u \in L : c|_{E(u)} \neq 0 \right\} \\ T &= \left\{ u \in R : c|_{E(u)} \neq 0 \right\} \end{aligned}$$

Using $\text{dist}(C_0)$ is large, and G is ε -pseudorandom, show that $\sqrt{|S| |T|}$ is large. Finish the proof!

□

9.2 Expander graphs

In the rest of the chapter, we will see how to construct ε -psuedorandom bipartite graphs. Let $G = (V, E)$ be a graph. Its adjacency matrix $A(G)$ is an $n \times n$ real symmetric matrix given by

$$[A(G)]_{u,v} = \begin{cases} 1 & \text{if } (u,v) \in E \\ 0 & \text{otherwise} \end{cases}$$

Recall that real symmetric matrices have real eigenvalues. Therefore, $A(G)$ has real eigenvalues $\lambda_1 \geq \lambda_2 \geq \dots \geq \lambda_n$. Moreover, we use the following fact without proof.

Fact 9.6. *If G is r -regular, then the largest eigenvalue of $A(G)$, denoted by $\lambda_1 = r$, and the corresponding eigenvector is all ones vector.*

This motivates the following definition of a spectral expander.

Definition 9.7. *We say a graph G is λ -spectral expander if*

$$\lambda = \max_{i \neq 1} \{|\lambda_i|\} = \max \{|\lambda_2|, |\lambda_n|\}.$$

Since, Tanner codes are based on bipartite graphs, we would be interested in “double cover” of graphs.

Definition 9.8. *The double cover of a graph $G = (V, E)$ is a bipartite graph (L, R, E') with*

1. *vertex set $L \cup R$ consists of two copies of vertices V : the left vertices $L = \{u_L : u \in V\}$ and right vertices $R = \{u_R : u \in V\}$.*
2. *and there is an edge $e = (u_L, v_R) \in E'$ if and only if $(u, v) \in E$.*

TODO: Draw double cover for a graph.

And we can use spectral expansion to build pseudorandom double covers. We follow the proof from [1].

Fact 9.9. *If G is λ -spectral expander, its double cover is λ/r -pseudorandom.*

Proof. **TODO:** Consider the spectral decomposition of adjacency matrix $A(G)$

$$A(G) = \sum_{i=1}^n \lambda_i |v_i\rangle\langle v_i|$$

Denote its double cover by (L, R, E) . Let $S \subset L$ and $T \subset R$, and define

$$|S\rangle = \sum_{i \in S} |i\rangle \quad \text{and} \quad |T\rangle = \sum_{i \in T} |i\rangle .$$

Prove that the edges $E(S, T)$ are given by

$$|E(S, T)| = \langle S | A(G) | T \rangle$$

Finish the proof!

□

Recall that to get good distance for Tanner code $\text{TC}(G, C_0)$ with ε -pseudorandom G , we need ε to be small. How small can we make $\varepsilon = \lambda/r$? Alon and Bopanna [3] showed that: if G is r -regular, then $\lambda \geq 2\sqrt{r-1} - o(1)$, where $o(1)$ refers to the asymptotic behaviour as $n \rightarrow \infty$ and r is fixed. This bound is tight for Ramanujan graphs.

Definition 9.10. We say a r -regular graph G is Ramanujan if $\lambda \leq 2\sqrt{r-1}$.

Definition 9.11. Let G be a group, and A be any subset of G . Then, the left Cayley graph $\text{Cay}_L(G, A) = (V, E)$ is defined as follows. The vertex set consists of a copy of group G : $V = G$. The edges are pairs of the form (g, ag) for every $g \in G$ and $a \in A$.

TODO: Draw a Cayley graph.

There are explicit families of Ramanujan graphs of degree $q + 1$ when q is any prime, proved independently by Margulis [4] and Lubotzky, Phillips and Sarnak [5]. These constructions are Cayley graphs, and their proofs are based on analogue of Ramanujan's conjecture.

Chapter 10

Quantum Tanner Codes

In this lecture, we will go over quantum Tanner codes, discovered by Leverrier and Zemor [6]. These are our good quantum LDPC codes i.e. $[[n, \Omega(n), \Omega(n)]]$ quantum error correcting codes with constant size parity checks. We will follow the analysis from a more recent paper from the same authors [7], and the discussion from [1].

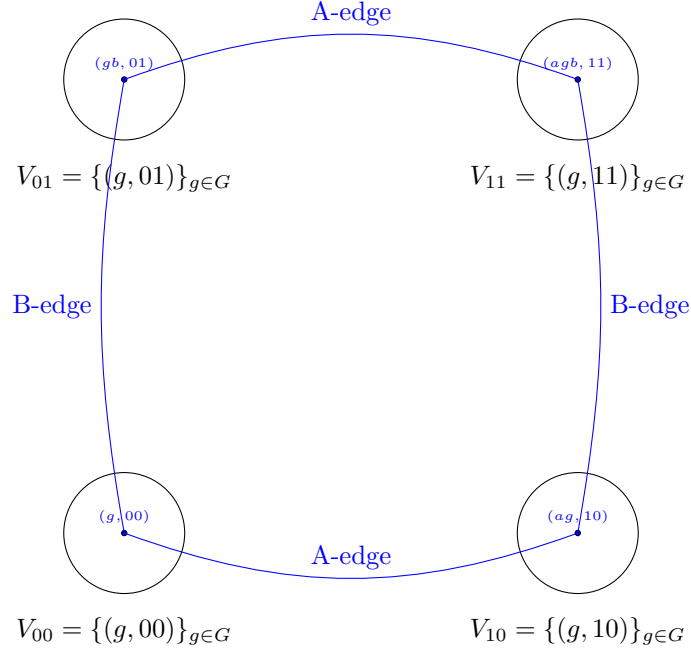
10.1 Left-Right Cayley Complex

The construction of quantum Tanner codes is based on left-right Cayley complex which was also used for constructing c^3 locally testable codes by Dinur et al. [8].

Definition 10.1 (Left-Right Cayley Complex, $\text{Cay}^2(A, G, B)$). *Let G be a group, and $A = A^{-1}, B = B^{-1}$ be two sets of generators¹. Assume $|A| = |B| = \Delta$. The Left-Right Cayley Complex $\text{Cay}^2(A, G, B)$ is defined as follows. It is made up of vertices, A -edges, B -edges and squares. The vertex set consists of 4 copies of the group: $V = V_{00} \cup V_{01} \cup V_{10} \cup V_{11}$ with $V_{ij} = G \times \{ij\}$. The A -edges are pairs of the form $\{(g, 00), (ag, 10)\}$ or $\{(gb, 01), (agb, 11)\}$. Similarly, the B -edges are pairs of the form $\{(g, 00), (gb, 01)\}$ or $\{(ag, 10), (agb, 11)\}$. A square is a set of the form $\{(g, 00), (ag, 10), (gb, 01), (agb, 11)\}$.*

TODO: Draw a Left-Right Cayley Complex with vertices, edges and squares.

¹A generator is a subset of the group such that every element of the group can be generated as a combination of elements of the generator, and their inverses.

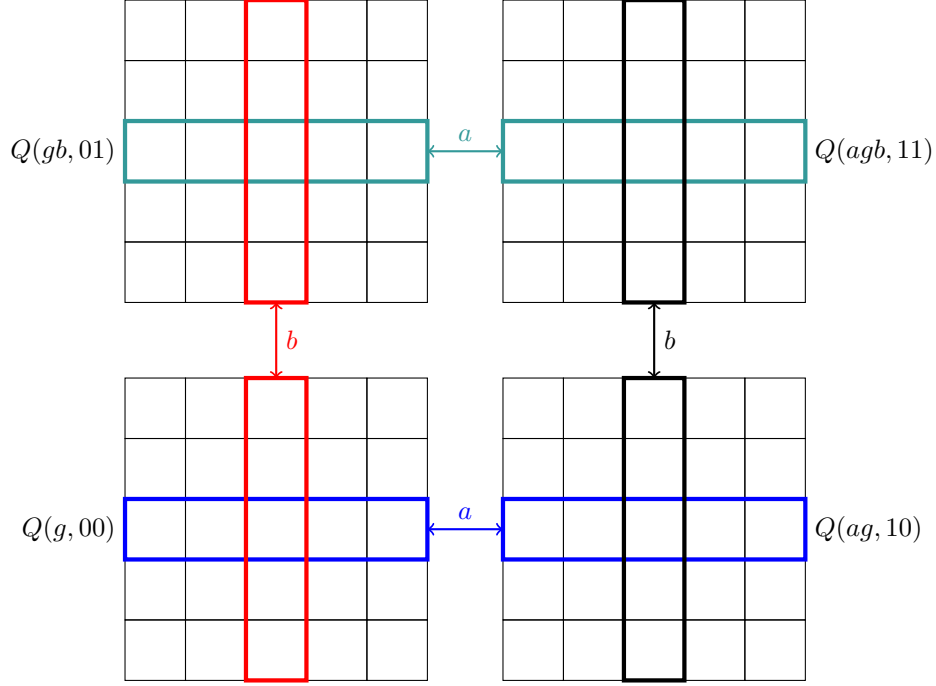


We denote the set of all squares by Q . Given a vertex $v \in V$, we would be interested in its Q-neighbourhood given by

$$Q(v) = \{\text{squares incident to } v\}$$

Note that for a fixed vertex v , every choice of $a \in A$ and $b \in B$, generates a unique square, for example for $v = (g, 00)$, the generated square would be $\{(g, 00), (ag, 10), (gb, 01), (agb, 11)\}$. Therefore, Q-neighborhood for any vertex v has cardinality Δ^2 and is isomorphic to $A \times B$. Therefore, we can view the Q-neighbourhood as a $\Delta \times \Delta$ grid. It turns out each column and row of this grid/matrix, is shared with Q-neighbourhoods of other vertices.

TODO: Draw the Q-neighbourhood for a vertex $(g, 00)$ rows indexed by $a \in A$ and columns indexed by $b \in B$. Show which row is shared with $Q(ag, 10)$ and which column is shared with $Q(gb, 01)$. Note that square $\{(g, 00), (ag, 01), (agb, 11), (gb, 10)\}$ is contained in $Q(g, 00)$, $Q(ag, 10)$, $Q(gb, 01)$ and $Q(agb, 11)$.



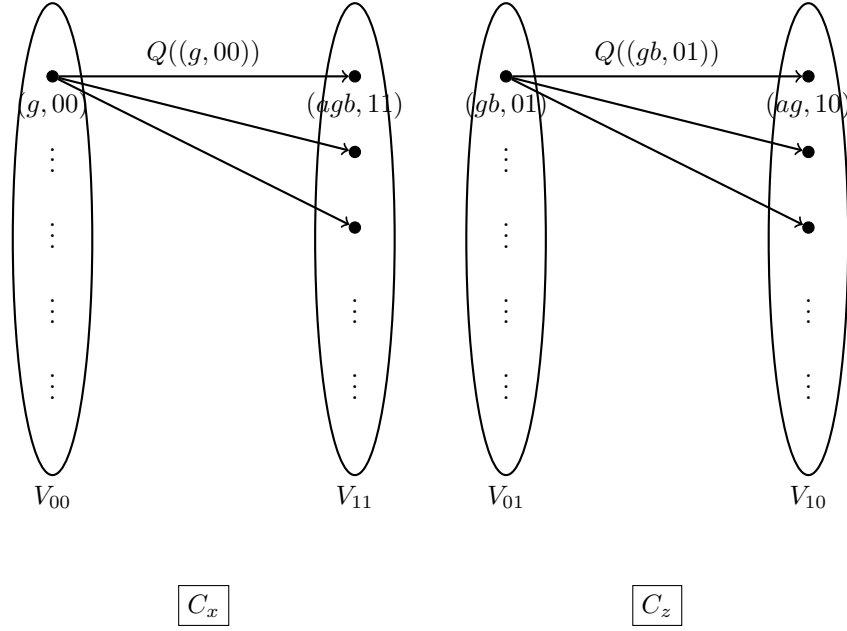
10.2 Quantum Tanner Code

In this section, we define Quantum Tanner codes, which are informally a higher dimensional generalization of classical Tanner codes. Using the left-right Cayley complex, the qubits will be associated to the squares of the complex, and the two bipartite graphs (embedded in the complex) will represent constraints on the squares.

Formally, Quantum Tanner Codes are special cases of CSS codes, whose underlying classical codes C_X and C_Z are both classical Tanner codes². We first define the bipartite graphs $\mathcal{G}_0^\square$ and $\mathcal{G}_1^\square$ for C_X and C_Z codes respectively. Both of these bipartite graphs are natural restrictions of the left right Cayley complex. If we restrict to vertices $V_0 = V_{00} \cup V_{11}$ (parity 0), every square is incident to only two vertices: one in V_{00} and another in V_{11} . Therefore, each square in Q can be thought of as an edge on V_0 . This gives us a natural bipartite graph $\mathcal{G}_0^\square = (V_{00}, V_{11}, Q)$. Similarly, if we restrict to vertices $V_1 = V_{10} \cup V_{01}$ (parity 1), we can define a bipartite graph $\mathcal{G}_1^\square = (V_{10}, V_{01}, Q)$.

²Recall that classical Tanner codes are based on bipartite graphs, where the global code lives on the edges, and every vertex has a local view of the global code, which is restricted to be from a base code.

TODO: Draw the bipartite graphs corresponding to the left-right Cayley complex.



The only other component we need to define the classical Tanner codes is the base code for the local view of a vertex v , which is given by the Q -neighbourhood $Q(v)$. Since, Q -neighbourhood is isomorphic to a $\Delta \times \Delta$ grid, we will use a product code as a base code.

Definition 10.2 (Product code). Let $C_A \subset \mathbb{F}_2^A$ be the linear error correcting code on $|A| = \Delta$ bits. Let $C_B \subset \mathbb{F}_2^B$ be the linear error correcting code on $|B| = \Delta$ bits. Then, the product code on $C_A \otimes C_B \subset \mathbb{F}_2^{A \times B}$ is the set of matrices x such that every column $(x_{ab})_{a \in A} \in C_A$ and every row $(x_{ab})_{b \in B} \in C_B$.

And from the definition of product code, the parity checks are generated

by row-checks, which verify if the rows are from C_B , and column-checks, which verify if the columns are from C_A . This gives us the dual to the product code.

Fact 10.3.

$$(C_A \otimes C_B)^\perp = C_A^\perp \otimes \mathbb{F}_2^B + \mathbb{F}_2^A \otimes C_B^\perp$$

It can also be checked that the generator for $C_A \otimes C_B$ is $G_A \otimes G_B$ where G_A, G_B are generators for C_A, C_B respectively. Based on the above discussion, we now formally define the classical codes C_X and C_Z for the quantum Tanner Code.

Definition 10.4 (X-code C_X).

$$C_X = \text{TC}(\mathcal{G}_0^\square, C_A^\perp \otimes \mathbb{F}_2^B + \mathbb{F}_2^A \otimes C_B^\perp)$$

Here, the variables are: Q . The parity checks are: for every $v \in V_0$, the bits on $Q(v)$ satisfies the parity checks $C_A \otimes C_B$. And the local code is: for every $v \in V_0$, bits on $Q(v)$ are from $C_A^\perp \otimes \mathbb{F}_2^B + \mathbb{F}_2^A \otimes C_B^\perp$.

TODO: Show how the local code C_X look like on a vertex neighbourhood.

Definition 10.5 (Z-code C_Z).

$$C_Z = \text{TC}(\mathcal{G}_1^\square, C_A \otimes \mathbb{F}_2^B + \mathbb{F}_2^A \otimes C_B)$$

Similar to X-code, the variables are: Q . The parity checks are: for every $v \in V_1$, the bits on $Q(v)$ satisfies the parity checks $C_A^\perp \otimes C_B^\perp$. And the local code is: for every $v \in V_0$, bits on $Q(v)$ are from $C_A \otimes \mathbb{F}_2^B + \mathbb{F}_2^A \otimes C_B$.

TODO: Show how the local code C_Z look like on a vertex neighbourhood.

Definition 10.6 (Quantum Tanner Code). *Let G be a group, and $A = A^{-1}$, $B = B^{-1}$ be two sets of generators. Assume $|A| = |B| = \Delta$. Let $C_A \subset \mathbb{F}_2^A$ and $C_B \subset \mathbb{F}_2^B$ be two linear error correcting code on Δ bits. Then, the corresponding quantum Tanner code $\text{QTC}(G, A, B, C_A, C_B)$ is the CSS code formed by the classical Tanner codes C_X and C_Z defined above.*

Using this, we can show that the CSS code formed by the classical codes C_X and C_Z is non-empty.

Fact 10.7. C_X and C_Z form a non-empty CSS code.

Proof. **TODO: Finish the proof!**

□

10.3 Dimension of the code

To get a lower bound on the dimension of our code, we need to compute the number of physical qubits, and the number of constraints/parity checks.

Fact 10.8. *Suppose C_A, C_B are $[\Delta, \rho\Delta]$ and $[\Delta, (1-\rho)\Delta]$ linear error correcting codes. Then, the quantum tanner code $\text{QTC}(G, A, B, C_A, C_B)$ is a $[[|G|\Delta^2, \geq |G|\Delta^2(1-2\rho)^2]]$ code.*

Proof. We first count the number of physical qubits, which is equal to number of squares. Recall that each triplet (g, a, b) for $g \in G$, $a \in A$, and $b \in B$ uniquely identifies a square. Therefore, the number of (distinct) squares are $|G|\Delta^2$, which gives the number of physical qubits.

Next, we want to count the number of independent parity checks. Lets start with parity checks from the X-code. These are elements of $C_A \otimes C_B$ for each vertex $v \in V_{00} \cup V_{11}$. Similarly, the parity checks for Z-code are elements of $C_A^\perp \otimes C_B^\perp$ for each vertex $v \in V_{01} \cup V_{10}$. Therefore the number of independent

parity checks are

$$\begin{aligned} &\leq 2|G| \dim(C_A \otimes C_B) + 2|G| \dim(C_A^\perp \otimes C_B^\perp) \\ &= 2|G| \dim(C_A) \dim(C_B) + 2|G| \dim(C_A^\perp) \dim(C_B^\perp) \end{aligned}$$

Recall C_A, C_B are $[\Delta, \rho\Delta]$ and $[\Delta, (1-\rho)\Delta]$ linear error correcting codes, which implies $\dim(C_A) = \rho\Delta$ and $\dim(C_A^\perp) = (1-\rho)\Delta$. Therefore, the number of independent parity checks

$$\leq 4|G| \rho(1-\rho)\Delta^2$$

And the dimension of the corresponding CSS code is

$$\begin{aligned} &\geq |G| \Delta^2 - 4|G| \rho(1-\rho)\Delta^2 \\ &= |G| \Delta^2 (1 - 4\rho(1-\rho)) \\ &= |G| \Delta^2 (1 - 2\rho)^2 \end{aligned}$$

□

Chapter 11

Distance of Quantum Tanner Code

In this lecture, we will show how to build a quantum Tanner code which is a $[[m, \Omega(m), \Omega(m)]]$ code for all m .

We start with classical codes C_A, C_B such that (a) C_A, C_B are $[\Delta, \rho\Delta]$ and $[\Delta, (1 - \rho)\Delta]$ linear error correcting codes, and (b) $C_A, C_B, C_A^\perp, C_B^\perp$ all have distance $\geq \delta\Delta$. It can be shown that there exists classical codes with constant $\rho, \delta \in (0, 1)$ for every value of Δ by checking that a random code have constant distance and rate. Then, similar to classical tanner code constructions, we pick G, A, B such that $\text{Cay}_L(G, A)$ and $\text{Cay}_R(G, B)$ are Ramanujan i.e. $\lambda(\text{Cay}_L(G, A)), \lambda(\text{Cay}_R(G, B)) \leq 2\sqrt{\Delta}$. This implies that the graphs $\mathcal{G}_0^\square, \mathcal{G}_1^\square$ are pseudorandom.

Fact 11.1 ([6]). $\mathcal{G}_0^\square, \mathcal{G}_1^\square$ are $4/\Delta$ -pseudorandom.

Proof. **TODO:** Here is a rough proof. $\mathcal{G}_0^\square$ is isomorphic to a double cover of graph $\bar{\mathcal{G}}_0^\square$ where vertices of $\bar{\mathcal{G}}_0^\square$ are all elements of group G , and there is an edge (g, agb) for every $g \in G$ and $a \in A$ and $b \in B$. Therefore, to prove our claim, using Fact 9.9, we only need to show that $\lambda(\bar{\mathcal{G}}_0^\square) \leq 4\Delta$ as degree of $\bar{\mathcal{G}}_0^\square$ is Δ^2 . To prove this, note that the adjacency matrix M of $\bar{\mathcal{G}}_0^\square$ is product of adjacency matrix M_A of $\text{Cay}_L(G, A)$ and adjacency matrix M_B of $\text{Cay}_R(G, B)$. By the structure of our complex, M_A and M_B commute i.e. $M_A M_B = M_B M_A$. Therefore, they share their eigenspaces, and the largest eigenvalues of M is Δ^2 , and absolute value of all other eigenvalues is $\leq 4\Delta$. Finish this proof!

□

Using Fact 10.8, we get that the number of physical qubits are $n = |G| \Delta^2$, and number of logical qubits are $k = n(1 - 2\rho)^2 = \Omega(n)$. Recall that to get good distance for classical Tanner codes, we only needed (a) distance of base code,

and (b) expansion of the underlying graph. But for quantum tanner codes we will need another property: product expansion.

11.1 Product expansion

We first informally discuss product expansion. Consider the Z-code $C_Z = \text{TC}(\mathcal{G}_1^\square, C_A \otimes \mathbb{F}_2^B + \mathbb{F}_2^A \otimes C_B)$. Let x be any codeword of C_Z . Recall that the Q-neighbourhood of a vertex $v \in V_1 = V_{01} \cup V_{10}$, denoted $Q(v)$, is a $\Delta \times \Delta$ grid. This means that the restriction of x to the neighbourhood $Q(v)$, denoted x_v , is an element of $C_A \otimes \mathbb{F}_2^B + \mathbb{F}_2^A \otimes C_B$. Equivalently, $x_v = c_v + r_v$ where $c_v \in C_A \otimes \mathbb{F}_2^B$ and $r_v \in \mathbb{F}_2^A \otimes C_B$. In other words, x_v is a matrix such that it is sum of matrices c_v and r_v where c_v is a matrix with columns in C_A , and r_v is a matrix with rows in C_B .

TODO: Draw the local view ($Q(v)$ neighbourhood of vertex $v \in V_1$) for a codeword $x \in C_Z$.

Define $\|c_v\|, \|r_v\|$ as the number of non-zero columns in c_v , and non-zero rows in r_v respectively. As we will see later on, expansion properties of our code will ensure that the number of non-zero columns/rows is large i.e. $\|c_v\| + \|r_v\|$ is large. Recall distance of C_A and C_B are both $\geq \delta\Delta$. This means, both the number of non-zero entries in c_v , and non-zero entries in r_v are large,

$$\begin{aligned} |c_v| &\geq \|c_v\| \delta\Delta \\ |r_v| &\geq \|r_v\| \delta\Delta \end{aligned}$$

At this point, ideally, we would like to say

$$|x_v| \approx |c_v| + |r_v| \geq (\|c_v\| + \|r_v\|)\delta\Delta$$

so that we can say that $|x_v|$, and in turn the distance of our code, is large. However, we can not in general lower bound the number of non-zero entries in x_v , as some of the 1's in c_v and r_v might add up to 0. This motivates the following definition:

Definition 11.2 (κ -product expansion). *For two linear codes $C_A, C_B \subset \mathbb{F}_2^\Delta$, the $C_A \otimes \mathbb{F}_2^B + \mathbb{F}_2^A \otimes C_B$ is said to be κ -product expanding, if any codeword $x \in C_A \otimes \mathbb{F}_2^B + \mathbb{F}_2^A \otimes C_B$ can be written as $x = c + r$ for some $c \in C_A \otimes \mathbb{F}_2^B$ and $r \in \mathbb{F}_2^A \otimes C_B$ such that*

$$|c + r| \geq \kappa \Delta (\|c\| + \|r\|)$$

where $\|c\|, \|r\|$ are the number of non-zero columns in c , and non-zero rows in r respectively

It turns out that random codes also have good product expansion.

Theorem 11.3 ([9]). *Fix $\rho \in (0, 1)$. Pick a uniformly random pair of linear codes C_A and C_B of size Δ , and dimensions $\rho\Delta, (1 - \rho)\Delta$ respectively. Then, as $\Delta \rightarrow \infty$, with high probability, the code $C_A \otimes \mathbb{F}_2^B + \mathbb{F}_2^A \otimes C_B$ is a κ -product expanding for*

$$\kappa = \frac{1}{2} \min \left\{ \frac{1}{4} H^{-1} \left(\frac{\rho}{8} \right) H^{-1} \left(\frac{1 - \rho}{8} \right), H^{-1} \left(\frac{\rho(1 - \rho)}{8} \right) \right\}$$

11.2 Minimal representation

Let $x \in C_Z - C_X^\perp$. We want to lower bound the weight of x . Recall C_Z is a Tanner code $C_Z = \text{TC}(\mathcal{G}_1^\square, C_A \otimes \mathbb{F}_2^B + \mathbb{F}_2^A \otimes C_B)$ where $\mathcal{G}_1^\square$ has vertex set $V_{01} \cup V_{10}$. The set of squares Q can be partitioned into Q -neighbourhoods $Q(v)$ for $v \in V_{01}$. As discussed before, on the neighbourhood $Q(v)$, x reduces to $x_v = c_v + r_v$ where c_v is a $C_A \otimes \mathbb{F}_2^B$ codeword and r_v is a $\mathbb{F}_2^A \otimes C_B$ codeword. We group the column vectors and row vectors and write

$$C_0 = \sum_{v \in V_{01}} \bar{c}_v \quad \text{and} \quad R_1 = \sum_{v \in V_{01}} \bar{r}_v$$

where $\bar{c}_v \in \{0, 1\}^Q$ is equal to c_v on $Q(v)$ and 0 everywhere else, and $\bar{r}_v \in \{0, 1\}^Q$ is defined similarly. And because the set of squares Q can be partitioned into Q -neighbourhoods $Q(v)$ for $v \in V_{01}$, C_0 is just concatenation of all the c_v for $v \in V_{01}$. Therefore,

$$C_0 + R_1 = x.$$

Similarly, by partitioning Q into $Q(v)$ neighbourhoods for $v \in V_{10}$, we define

$$C_1 = \sum_{v \in V_{10}} \bar{c}_v \quad \text{and} \quad R_0 = \sum_{v \in V_{10}} \bar{r}_v$$

which again satisfy

$$C_1 + R_0 = x.$$

Note that definition of C_0, C_1, R_0, R_1 depend on the decomposition $x_v = c_v + r_v$ for each vertex $v \in V_1 = V_{01} \cup V_{10}$. Since, there are several possible decompositions of the local view x_v , there are many possible representations (C_0, C_1, R_0, R_1) for the codeword x .

Definition 11.4. We say (C_0, C_1, R_0, R_1) is a minimal representation of x if it minimizes the quantity $\|C_0\| + \|C_1\| + \|R_0\| + \|R_1\|$ where $\|C_0\| = \sum_{v \in V_{01}} \|c_v\|$ is the total number of non-zero columns in $\{c_v\}_{v \in V_{01}}$, and $\|C_1\|, \|R_0\|, \|R_1\|$ are defined similarly. We define the minimum value of $\|C_0\| + \|C_1\| + \|R_0\| + \|R_1\|$ as the norm of x , denoted as $\|x\|$.

Since $C_0 + R_1 = x = C_1 + R_0$, we get that

$$\begin{aligned} C_0 + R_1 + C_1 + R_0 &= 0 \\ \implies C_0 + R_0 &= C_1 + R_1 \end{aligned}$$

Define $x^0 \stackrel{\text{def}}{=} C_0 + R_0 = C_1 + R_1$, and therefore (C_0, C_1, R_0, R_1) is also a representation for x^0 . Moreover, we can show that x^0 restricted to Q -neighborhood $Q(v)$ for vertex $v \in V_0$ is a codeword of $C_A \otimes \mathbb{F}_2^B + \mathbb{F}_2^A \otimes C_B$.

Fact 11.5. $x^0 \in \text{TC}(\mathcal{G}_0^\square, C_A \otimes \mathbb{F}_2^B + \mathbb{F}_2^A \otimes C_B)$

Proof. Recall that

$$C_0 = \sum_{v \in V_{01}} \bar{c}_v$$

where $\bar{c}_v \in \{0, 1\}^Q$ is equal to c_v on $Q(v)$ and 0 everywhere else. So, the restriction $C_0|_{Q(v)}$ for $v \in V_{01}$ is a matrix whose each column is a codeword in $C_A \otimes \mathbb{F}_2^B$.

The Q -neighborhood $Q(g, 00)$ of a vertex $(g, 00) \in V_{00}$ is a $\Delta \times \Delta$ grid, with rows indexed by $a \in A$, and columns indexed by $b \in B$. Moreover, the column indexed by b is shared with the Q -neighbourhood of $(gb, 01) \in V_{01}$. This implies the restriction $C_0|_{Q(v)}$ for $v \in V_{00}$ is also a matrix whose each column is a codeword in $C_A \otimes \mathbb{F}_2^B$.

Similarly, the restriction $R_0|_{Q(v)}$ for $v \in V_{00}$ is a matrix whose each row is a codeword in $\mathbb{F}_2^A \otimes C_B$. Together, this implies their sum, $C_0 + R_0|_{Q(v)}$ for $v \in V_{00}$ is a codeword in $C_A \otimes \mathbb{F}_2^B + \mathbb{F}_2^A \otimes C_B$, which completes the proof. $\square$

(C_0, C_1, R_0, R_1) is a representation for both x and x^0 . Now, if (C_0, C_1, R_0, R_1) is a minimal representation for x , it need not be a minimal representation for x^0 . However, we can prove this is true for special x .

Fact 11.6. If x has minimum norm in coset $x + C_X^\perp$, then a minimum representation for x is also a minimum representation for x^0 .

Proof. **TODO: Finish the proof!**

□

11.3 Minimum Distance

In this section, we will prove the following lemma:

Lemma 11.7. *For any $x \in C_Z - C_X^\perp$, we have*

$$\|x\| \geq \frac{\delta^2 \kappa n}{512 \Delta^2} = \Omega(n)$$

This will be enough to prove our main theorem: a lower bound on the minimum distance of quantum Tanner codes.

Theorem 11.8.

$$d_Z^+ = \min_{x \in C_Z - C_X^\perp} |x| \geq \frac{\delta^2 \kappa^2 n}{1024 \Delta}$$

Proof. Consider $x \in C_Z - C_X^\perp$. Recall C_Z is a Tanner code $C_Z = \text{TC}(\mathcal{G}_1^\square, C_A \otimes \mathbb{F}_2^B + \mathbb{F}_2^A \otimes C_B)$. This means, the restriction x_v of x on the Q -neighbourhood $Q(v)$ for some vertex $v \in V_{01}$ is a codeword of $C_A \otimes \mathbb{F}_2^B + \mathbb{F}_2^A \otimes C_B$. Consider the minimal representation (C_0, C_1, R_0, R_1) . Then, for a vertex $v \in V_{01}$, the decomposition of x_v into $c_v \in C_A \otimes \mathbb{F}_2^B$ and $r_v \in \mathbb{F}_2^A \otimes C_B$ satisfies (as minimality of the representation implies product expansion)

$$x_v = c_v + r_v \quad \text{and} \quad |c_v + r_v| \geq \kappa \Delta (\|c_v\| + \|r_v\|)$$

The representation (C_0, C_1, R_0, R_1) also satisfies

$$\begin{aligned} |x| &= \sum_{v \in V_{01}} |\bar{c}_v + \bar{r}_v| \\ &= \sum_{v \in V_{01}} |c_v + r_v| \\ &\geq \sum_{v \in V_{01}} \kappa \Delta (\|c_v\| + \|r_v\|) \\ &= \kappa \Delta (\|C_0\| + \|R_1\|) \end{aligned}$$

where the first step follows because $Q(v)$ for $v \in V_{01}$ (or in fact any V_{ij}) forms a partition of set of all squares Q , the second step follows because $\bar{c}_v, \bar{r}_v$ are 0 outside of $Q(v)$ and equal to c_v, r_v inside $Q(v)$ respectively, the third step follows from κ -product expansion, and the last step follows from definition of $\|C_0\|$ and $\|R_0\|$. Similarly, we get

$$|x| \geq \kappa \Delta (\|C_1\| + \|R_0\|)$$

and therefore,

$$|x| \geq \frac{\kappa\Delta}{2}(\|C_0\| + \|C_1\| + \|R_0\| + \|R_1\|) = \frac{\kappa\Delta}{2} \|x\|.$$

□

It remains to prove Lemma 11.7. Let (C_0, C_1, R_0, R_1) be a minimal representation of x . We first identify the set of vertices whose local view is non-zero. Define

$$S_{ij} = \{v \in V_{ij} : C_i + R_j \text{ is non zero on } Q(v)\}$$

so S_{01} are the vertices v in V_{01} such that $x|_{Q(v)} \neq 0$, S_{10} are the vertices v in V_{10} such that $x|_{Q(v)} \neq 0$, and so on. Set $S_0 = S_{00} \cup S_{11}$ and $S_1 = S_{01} \cup S_{10}$.

Definition 11.9 (Ordinary and Exceptional Vertices). *We say $v \in S_{ij}$ is exceptional if it has many non-zero columns and rows, $\|c_v\| + \|r_v\| \geq \alpha\Delta$ for $\alpha = \delta^2/256$ where c_v and r_v are restrictions of C_i and R_j on neighbourhood $Q(v)$ respectively. Otherwise, we say v is ordinary. We denote these exception vertices in S_{ij} by S_{ij}^e and ordinary vertices in S_{ij} by S_{ij}^o .*

We can show that whenever S_{ij} is small enough, the number of exceptional vertices is a small fraction of $|S_{ij}|$, of order $O(1/\Delta^2)$.

Lemma 11.10. *If $S_{00}, S_{01}, S_{10}, S_{11} \leq \frac{\kappa\alpha}{2} |V_{00}|$, then*

$$\begin{aligned} |S_{00}^e|, |S_{11}^e| &\leq \frac{64}{\alpha^2 \kappa^2 \Delta^2} |S_0| \\ |S_{10}^e|, |S_{01}^e| &\leq \frac{64}{\alpha^2 \kappa^2 \Delta^2} |S_1| \end{aligned}$$

Proof. We prove the upper bound for S_{00}^e . Recall that $\mathcal{G}_0^\square$ is a $4/\Delta$ pseudorandom bipartite graph (Fact 11.1), where S_{00}^e is a subset of left vertices and S_{11} is a subset of right vertices. Using definition of pseudorandomness, Definition 9.4, we get that

$$|E(S_{00}^e, S_{11})| \leq \frac{\Delta^2}{|V_{00}|} |S_{00}^e| |S_{11}| + \frac{4}{\Delta} \Delta^2 \sqrt{|S_{00}^e| |S_{11}|}$$

We can lower bound the number of edges $|E(S_{00}^e, S_{11})|$ by $|S_{00}^e| \times$ the minimum number of non-zero squares in Q -neighbourhood $Q(v)$ of vertex $v \in S_{00}^e$. The later can be lower bounded by definition of product expansion, and exceptional vertices. Together, we obtain

$$|E(S_{00}^e, S_{11})| \geq \kappa\alpha\Delta^2 |S_{00}^e|$$

. Further, using our assumption, $|S_{11}| / |V_{00}| \leq \kappa\alpha/2$, we obtain

$$\begin{aligned} \kappa\alpha\Delta^2 |S_{00}^e| &\leq |E(S_{00}^e, S_{11})| \\ &\leq \frac{\kappa\alpha\Delta^2}{2} |S_{00}^e| + 4\Delta \sqrt{|S_{00}^e| |S_{11}|} \\ \implies \frac{\kappa\alpha\Delta^2}{2} |S_{00}^e| &\leq 4\Delta \sqrt{|S_{00}^e| |S_{11}|} \end{aligned}$$

which completes the proof as $S_{11} \subset S_0$. □

We need one more definition. Suppose $|S_1| \geq |S_0|$ and $|S_{01}| \geq |S_{10}|$. The other cases are similarly handled. Recall that $v = (g, 00)$ and $u = (gb, 01)$ share a column of c_v , in particular the column corresponding to $b \in B$. Based on this, we define

$$T = \{v \in S_{00} : \text{a non-zero column of } c_v \text{ is shared with an ordinary } v \in S_{01}^o\}$$

We show that whenever S_{ij} is small, the number of vertices in T is also small.

Lemma 11.11. *If $|S_{10}| \leq \delta |V_{00}|/4$, then*

$$|T| \leq \frac{64}{\delta^2 \Delta} |S_{10}|$$

Proof. If we restrict ourselves to vertices V_{00} and V_{10} , then every square of the square complex is reduced to an A-edge. This bipartite graph on $V_{00} \cup V_{10}$ is exactly the double cover of $\text{Cay}(G, A)$.

Similar to before, we first lower bound $|E(T, S_{10})|$ by $|T| \times$ the minimum number of non-zero squares in Q-neighbourhood $Q(v)$ of vertex $v \in T$. Now, the minimum number of non-zero squares in Q-neighbourhood $Q(v)$ of vertex $v \in T$ is at least $\delta\Delta/2$. This is because each non-zero column in c_v is a codeword in $\mathbb{F}_2^A \times C_B$, and therefore has at least $\delta\Delta$ ones. To compute $|x_v|$, note that by definition of T , $v \in T$ is ordinary i.e. it has at most $(\delta^2/256)\Delta$ non-zero rows. This implies

$$|x_v| \geq \delta\Delta - (\delta^2/256)\Delta \geq \delta\Delta/2$$

In turn, this implies

$$|T| \delta\Delta/2 \leq |E(T, S_{10})|$$

Also, since $\lambda(\text{Cay}(G, A)) \geq 2\sqrt{\Delta}$, its double cover of $\text{Cay}(G, A)$ is $2\sqrt{\Delta}/\Delta$ -pseudorandom Fact 9.9. So applying the definition of pseudorandomness, Definition 9.4, we get

$$|E(T, S_{10})| \leq \Delta \frac{|T| |S_{10}|}{|V_{10}|} + 2\sqrt{\Delta} \sqrt{|T| |S_{10}|}$$

And finally using our assumption that $|S_{10}| \leq \delta |V_{00}|/4$ (and as $|V_{00}| = |V_{10}|$), we get

$$|E(T, S_{10})| \leq \frac{\Delta\delta}{4} |T| + 2\sqrt{\Delta} \sqrt{|T| |S_{10}|}$$

which completes the proof, same as the last proof. $\square$

Lemma 11.10 tells us that there are very few exceptional vertices, whereas as we will see Lemma 11.11 tells us that because columns shared with ordinary vertices are clustered together, there will be many exceptional vertices, leading to a contradiction. We prove this formally.

Proof. The proof is by contradiction. Suppose $x \in C_Z - C_X^\perp$ satisfies $\|x\| < \kappa\delta^2 n/512\Delta^2$. Note that we can assume x has the minimum norm $\|x\|$ in $x + C_X^\perp$, and therefore can assume that Fact 11.6 holds. Also, let (C_0, C_1, R_0, R_1) be a minimal representation of x . Since,

$$|S_{ij}| \leq \|C_i\| + \|R_j\| \leq \|C_0\| + \|C_1\| + \|R_0\| + \|R_1\| = \|x\| ,$$

we have that

$$|S_{ij}| < \kappa\delta^2/512 |V_{00}| = \frac{\kappa\alpha}{2} |V_{00}| \leq \frac{\delta}{4} |V_{00}| .$$

as $\alpha = \delta^2/256$, $|V_{00}| = |G| = n/\Delta^2$ and $\kappa\alpha/2 \leq \delta/4$. Therefore, both Lemma 11.10 and Lemma 11.11 hold. In particular, using Lemma 11.10, we get that

$$\begin{aligned} |S_{01}^e| &\leq \frac{64}{\alpha^2 \kappa^2 \Delta^2} |S_{01}| \\ \implies |S_{01}^o| &\geq a |S_{01}| \end{aligned}$$

for any constant $a \in (0, 1)$ and Δ large enough. Recall that v belongs to S_{01}^o implies at least one of the columns or rows in $Q(v)$ is non-zero. This means either the number of non-zero rows or number of non-zero columns (in the Q-neighbourhood) of ordinary vertices S_{01}^o is at least $a/2 |S_{01}|$. Without loss of generality, assume the number of non-zero columns of ordinary vertices is at least $a/2 |S_{01}|$.

Applying Lemma 11.11, the non-zero columns of ordinary vertices S_{01}^o must cluster among the Q-neighbourhoods of the vertices of the set T of size at most $64/\delta^2 \Delta |S_{10}|$. Therefore,

$$\begin{aligned} \text{avg}_{v \in T} \|c_v\| + \|r_v\| &\geq \frac{1}{T} \sum_{v \in T} \|c_v\| \\ &\geq \frac{1}{T} \frac{a}{2} |S_{01}| \geq \frac{\delta^2 \Delta a}{128} \end{aligned}$$

Recall v is ordinary if $\|c_v\| + \|r_v\| < \delta^2 \Delta/256$. Since, $\|c_v\|, \|r_v\| \leq \Delta$, we get that if p fraction of S_{01} is exceptional, then

$$p2\Delta + (1-p)\delta^2 \Delta/256 \leq \frac{\delta^2 \Delta a}{128}$$

Therefore, a constant fraction of T are exceptional vertices of S_{00} .

Now, because there are $a/2 |S_{01}^o|$ many ordinary columns from above, and a Q-neighbourhood of a vertex can host at most Δ columns,

$$|T| \geq \frac{1}{\Delta} \frac{a}{2} |S_{01}^o| \geq a/2\Delta |S_{01}| \geq a/4\Delta |S_1| ,$$

where the last step follows from $|S_{01}| \geq |S_{10}|$. Together with statement above, this gives that the number of exceptional vertices are

$$|S_{00}^e| \geq |T| p \geq 1/\Delta (\text{constant independent of } \Delta) |S_1| .$$

But also by Lemma 11.10,

$$|S_{00}^e| \leq 1/\Delta^2 (\text{constant independent of } \Delta) |S_1|$$

This is a contradiction for Δ sufficiently large.

□

Chapter 12

Floquet codes (Beginner's Guide)

Mini-chapter by Hanna Westerheim.

12.1 Learning Objectives/Author's Disclaimer

The goal of this mini-chapter is to give an accessible introduction to Floquet codes. To do so, I will be focusing on one type of Floquet code called the honeycomb code, or XYZ2 code, which is the original code of this type introduced by Hastings and Haah in 2021 [10]. This mini-chapter will attempt to give key take-aways about what Floquet codes are, as well as high-level intuition on how they work, with the target audience being those who are completely unfamiliar with their construction. If interested, the reader may find more technical details in the original source.

Section 12.2 provides necessary background and context from which Floquet codes are introduced. Section 12.3 defines the honeycomb Floquet code, and Section 12.4 attempts to provide some basic intuition about how the code works. Further details regarding the logical operators, decoding graph, boundary conditions, and more can be found in the original work.

12.2 Background

So far in this class, we have considered what are known as *subspace codes*. Namely, we've been working with the following definition: a quantum error correcting code is a subspace C of some larger physical Hilbert space H_{physical} . Every construction that we have seen in this class (toric codes, chain complexes, hypergraph product codes, tanner codes, etc.) has been an example of a subspace code.

However, this is not the most general notion for encoding quantum information. We can instead encode information into a subsystem of the physical quantum system. The first widely recognized *subsystem code* is considered to be the Bacon-Shor code. Without getting into the details, I will attempt to explain what is different between a subspace code and a subsystem code, taking the Bacon-Shor code as an example.

The key point here is that a subsystem code is basically a subspace code where we don't care about some of the logical qubits. Let's, for a moment, consider the Bacon-Shor code as a subspace code. We have a d by d lattice of physical qubits, and $2(d-2)$ independent stabilizer generators. Thus, when viewed as a subspace code, the Bacon-Shor code has $k = d^2 - 2(d-1)$ logical qubits.

The problem here is that only one of these logical qubits has distance d and the rest have distance two. If we “forget” about the logical qubits which only have distance two, we can preserve our code distance at d . There's an additional benefit here as well, which is a bit more subtle. By “not caring” about the logical degrees of freedom on the “gauge qubits” (the logical qubits which we have forgotten about), we can freely measure the two-qubit anti-commuting “gauge operators” (the logical operators on these gauge qubits). We can then use the values of these two-qubit measurements to infer the values of the larger stabilizers. In other words, this allows us to perform smaller measurements than we would have otherwise needed in a subspace code with same-weight stabilizer generators, which is both easier to do physically and better when considering circuit-level noise in the measurement circuits.

We just generalized from subspace codes to subsystem codes. From here, we can generalize again and arrive at what will be the core focus of this mini-chapter: Floquet codes. Now we will allow the subspace to change across different timesteps, i.e. at each timestep, I perform some set of anti-commuting measurements, which moves the information from one subspace to another. The key idea here is that although the subspace is changing at each timestep, we do so in a clever way such that logical information is preserved. Hastings and Haah introduced this idea in their 2021 paper [10]. We call such codes with dynamically generated logical qubits, Floquet codes. Nowadays, there are a number of Floquet code constructions, but here we will focus on the original: the honeycomb Floquet code.

12.3 The Honeycomb Floquet Code

Let's finally define the honeycomb Floquet code and draw some pictures. In the honeycomb code, we place qubits on the vertices of a hexagonal (or “honeycomb”) lattice as shown in Figure 12.1. Each of the figures in this mini-chapter will show only a small portion of the lattice. We can think of the lattice extending much farther in each direction and then eventually wrapping back around on itself on a torus (giving us periodic boundary conditions).

In order to properly define the code, we will need to label plaquettes and

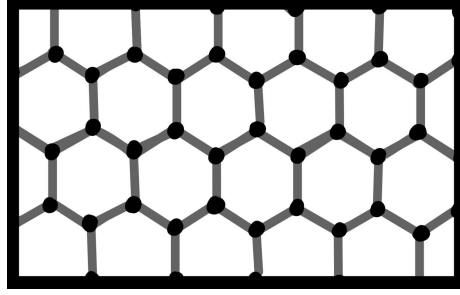


Figure 12.1: Honeycomb lattice. In the honeycomb code, qubits are placed on the vertices of a hexagonal (or honeycomb) lattice.

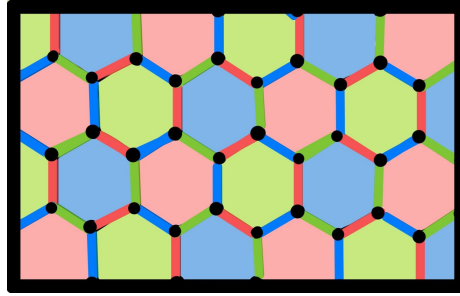


Figure 12.2: Edge and plaquette coloring. The plaquettes of the honeycomb lattice are three-colored. Edges are colored according to the color of the plaquettes at their endpoints, i.e. red edges connect red plaquettes.

edges in a very specific way. Let's three-color the graph and associate each color with a labeling of plaquettes. Figure 12.2 depicts the plaquette labeling scheme along with one of the labels for the edges (color).

Each edge will have two labels: a color (where each color corresponds to a unique number in the original paper), and a letter in x, y, z . The color of each edge is the color of the plaquettes it connects, i.e. red edges connect red plaquettes. Additionally, each edge will be labeled with a letter according to that edge's direction. A key defining how each edge direction corresponds to a labeling in x, y, z is given in Figure 12.3.

Executing the code requires a sequence of “edge measurements,” which we will call checks. For example, if we want to measure all “red checks,” this means we will measure a two-qubit operator on every edge which is labeled as red. The two-qubit operator corresponding to each edge is given by its x, y, z labeling. On edges labeled with an x , we will measure the two-qubit Pauli XX on the two qubits in the support of the check. Similarly, on edges labeled with y or z , we will measure YY or ZZ respectively.

Now that we have properly defined elements of the lattice, we can introduce

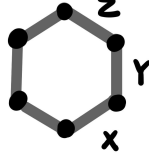


Figure 12.3: Edge labeling key. In addition to a color labeling, edges are labeled according to their direction with x , y , or z . This labeling corresponds to the type of two body measurement which is to be performed on this check. For example, edges labeled with x , undergo Pauli XX check measurements.

the sequence of checks which defines the honeycomb Floquet code. The procedure is simple and is periodic with period three. Below is the measurement cycle and the corresponding instantaneous stabilizer group (ISG) associated with each time step. It is important to note that there is a warm-up cycle, in which the stabilizer group grows over the three rounds. Here we will only consider the behavior in the steady state.

- Round 0 mod 3: Measure all red checks. The ISG is generated by the set of all red checks, and the set of all plaquette stabilizers.
- Round 1 mod 3: Measure all green checks. The ISG is generated by the set of all green checks, and the set of all plaquette stabilizers.
- Round 2 mod 3: Measure all blue checks. The ISG is generated by the set of all blue checks, and the set of all plaquette stabilizers.

A plaquette stabilizer is the product of all checks around a plaquette. Similar to subsystem codes, we do not actually have to measure these 6-body plaquette stabilizers and can instead infer their measurement outcomes from the two-body check measurements. More specifically, the plaquette stabilizer measurement values of a certain color can be inferred from the measurement outcomes from the current round and the previous round. For example, at round 1 mod 3, we can infer the value of the blue plaquette stabilizers from the green and red check measurement outcomes from the current and previous rounds respectively. This can be made clear by observing the edge and plaquette coloring in Figure 12.2. Blue plaquette stabilizers consist of only red and green checks (which form cycles around blue plaquettes).

The set of all plaquette stabilizers forms a static subgroup of the ISG, i.e. these stabilizers always exist throughout time. Notice that the only part that changes are the two-body check generators, which correspond to the checks just measured in each round.

This specific ordering of checks, although anti-commuting, preserves information throughout time. We can do some counting to determine how many logical qubits are present. Given that there are n_p plaquettes in the lattice, we can see that there are $2n_p$ physical qubits. There are $2n_p$ stabilizers in each ISG.

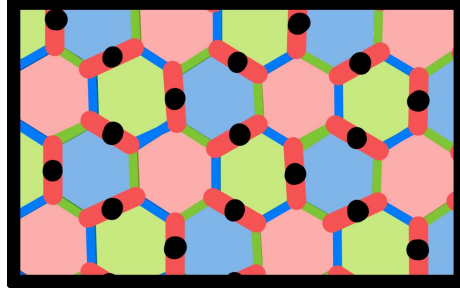


Figure 12.4: Round $0 \bmod 3$ check measurements (red checks). At round $0 \bmod 3$, we measure all red checks. The two qubits in the support of these checks lose a degree of freedom from this measurement and can thus be thought of as one effective qubit. These effective qubits are illustrated as large black dots.

However, multiplying all of the plaquette stabilizers together yields the identity and so does multiplying all of the checks with all of the plaquette stabilizers of the same color. Thus, each ISG has $2n_p - 2$ independent generators, and so there are $k = (2n_p) - (2n_p - 2) = 2$ logical qubits.

Another way to see that there are two logical qubits and to better understand the code in general, is by observing an equivalence between the ISG at each timestep and the toric code.

12.4 Instantaneous Toric Code Equivalence

Suppose that we are at round $0 \bmod 3$, and we just measured all red checks. Now the two qubits in the support of each check can be thought of as one effective qubit, as the check measurement removed a degree of freedom. These checks and their associated effective qubits are illustrated in Figure 12.4.

Now we can draw a hexagonal superlattice on which the effective qubits lie. This lattice is drawn in Figure 12.5

The red plaquette stabilizers will act as plaquette stabilizers in the superlattice, and the green and blue plaquette stabilizers will act as vertex stabilizers in the superlattice. We have thus realized an instantaneous toric code on the superlattice. It is easier to see now why there are two logical qubits, and we can additionally see that the distance will be proportional to the linear size of the torus.

In the next round, green checks will be measured, and a new instantaneous toric code will be realized on a shifted superlattice, where the roles of red, green, and blue plaquette stabilizers will shift as well. This pattern continues throughout the duration of the code.

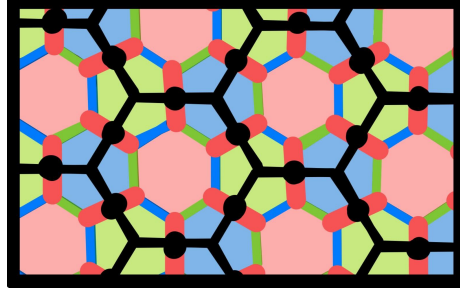


Figure 12.5: Effective qubit superlattice. The effective qubits in Figure 12.4 are drawn here on the edges of a hexagonal superlattice. The plaquette stabilizer group forms an instantaneous toric code on the superlattice.

12.5 Conclusion

In this mini-chapter, we saw the series of generalizations from subspace codes (which we have seen throughout this class) leading up to the introduction of Floquet codes. From there, the honeycomb Floquet code was introduced as the original Floquet code construction, and concretely defined in order to aid the reader's understanding. Finally, we took a deeper look into how the honeycomb code works, by observing an equivalence between the ISG and an instantaneous toric code on a superlattice.

Again, the goal of this mini-chapter was to give an accessible introduction to Floquet codes and a taste of how they work. The interested reader can find more technical details in the original work as well as more constructions and studies in the subsequent, increasingly rich Floquet codes literature.

Bibliography

- [1] John Wright. Lecture notes on quantum coding theory. <https://people.eecs.berkeley.edu/~jswright/quantumcodingtheory24/>, 2024.
- [2] Jean-Pierre Tillich and Gilles Zemor. Quantum ldpc codes with positive rate and minimum distance proportional to the square root of the blocklength. *IEEE Transactions on Information Theory*, 60(2):1193–1202, February 2014. ISSN 1557-9654. doi: 10.1109/tit.2013.2292061. URL <http://dx.doi.org/10.1109/TIT.2013.2292061>.
- [3] Noga Alon. Eigenvalues and expanders. *Combinatorica*, 6(2):83–96, 1986.
- [4] Grigori Aleksandrovich Margulis. Explicit group-theoretical constructions of combinatorial schemes and their application to the design of expanders and concentrators. *Problemy peredachi informatsii*, 24(1):51–60, 1988.
- [5] Alexander Lubotzky, Ralph Phillips, and Peter Sarnak. Ramanujan graphs. *Combinatorica*, 8(3):261–277, 1988.
- [6] Anthony Leverrier and Gilles Zémor. Quantum tanner codes, 2022. URL <https://arxiv.org/abs/2202.13641>.
- [7] Anthony Leverrier and Gilles Zémor. Decoding quantum tanner codes, 2022. URL <https://arxiv.org/abs/2208.05537>.
- [8] Irit Dinur, Shai Evra, Ron Livne, Alexander Lubotzky, and Shahar Mozes. Locally testable codes with constant rate, distance, and locality. In *Proceedings of the 54th Annual ACM SIGACT Symposium on Theory of Computing*, pages 357–374, 2022.
- [9] Gleb Kalachev and Pavel Panteleev. Two-sided robustly testable codes. *arXiv preprint arXiv:2206.09973*, 2022.
- [10] Matthew B. Hastings and Jeongwan Haah. Dynamically Generated Logical Qubits. *Quantum*, 5:564, October 2021. ISSN 2521-327X. doi: 10.22331/q-2021-10-19-564. URL <https://doi.org/10.22331/q-2021-10-19-564>.